Investment Research Agent: Multi-Agent Equity Research & Factual Grounding

---

Academic Information & Course Attribution

- **Institution:** University of San Diego (USD), Shiley-Marcos School of Engineering
- **Degree Program:** Master of Science in Applied Artificial Intelligence (MS-AAI)
- **Course: AAI-520-IN2: Natural Language Processing and Generative AI**
- **Instructor: Prof. Rajesh Soni, M.S.**
- **Project Repository:** https://github.com/diaeshantony-usd/investment-research-agent
- **Project Team:**
  1. **Sai Krishna Akula** (`nakula@sandiego.edu`)
  2. **Ashok Kumar Bhairwal** (`abhairwal@sandiego.edu`)
  3. **Diaesh Antony** (`dantony@sandiego.edu`)

What this notebook contains

This notebook explains **agent design and workflows**, **agent functions and capabilities**, and **evaluation and iteration**. We follow the order below, and every claim is backed by a cell that runs the real code from the repository.

Part	Sections	Rubric criterion
I. Agent design and workflows	1 Problem and design goals · 2 Architecture · 3 Data sources and tools · 4 Framework internals · 5 The agent team · 6 End-to-end run	Code (comments, visualisations)
II. Agent functions and capabilities	7 F1 Planning · 8 F2 Dynamic tool use · 9 F3 Self-reflection · 10 F4 Learning across runs · 11 Conversation memory	Agent Functions
III. Workflow patterns	12 W1 Prompt chaining · 13 W2 Routing · 14 W3 Evaluator-optimizer	Workflow Patterns
IV. Evaluation and iteration	15 Evaluation · 16 Iteration and code quality · 17 Summary	Code

Where each graded requirement is demonstrated

ID	Requirement	How this system meets it	Section
F1	Plans its research steps for a given stock symbol	Planner verifies the ticker and returns an ordered research plan as JSON	7

ID	Requirement	How this system meets it	Section
F2	Uses tools dynamically (APIs, datasets, retrieval)	Specialists pick tools and arguments at run time: Yahoo Finance, SEC EDGAR retrieval, FRED, NewsAPI	3, 6, 8
F3	Self-reflects to assess the quality of its output	Critic checks every figure against the raw data and scores the brief	9
F4	Learns across runs (keeps notes to improve future analyses)	Long-term memory: dated ticker notes and Critic lessons fed into the next run	10
W1	Prompt chaining: ingest, preprocess, classify, extract, summarize	News analyst's five-stage chain	12
W2	Routing to the right specialist (earnings, news, market)	Planner route + router that delegates to only the specialists a question needs	13
W3	Evaluator-optimizer: generate, evaluate, refine with feedback	Draft writer, Critic and Refiner loop with stopping rules	14

How to run this notebook

Option 1: Google Colab (nothing to install)

1. Open this link: https://colab.research.google.com/github/diaeshantony-usd/investment-research-agent/blob/main/notebooks/investment_research_agent.ipynb
2. Choose **Runtime** → **Run all**.
3. Click **Run anyway** when Colab warns that the notebook comes from GitHub.

The first cell clones the repository and installs it. A full run takes about 10–15 minutes, because it calls the LLM and the market-data APIs live.

Option 2: run it locally

```
git clone https://github.com/diaeshantony-usd/investment-research-agent.git
cd investment-research-agent
pip install -e .
jupyter notebook notebooks/investment_research_agent.ipynb
```

Setup

The first cell installs the project and dependencies (in Colab it clones the repository first; locally it automatically installs from `pyproject.toml` if dependencies are missing, or you can run `%pip install -e ..`), loads API keys, clears long-term memory so section 10 starts from scratch, and creates the shared LLM client. The second cell defines small display helpers used in the rest of the notebook.

In [1]:
```
# Setup: works both in Google Colab and locally.
# In Colab the repository is cloned and installed; locally dependencies are checked
# and installed from pyproject.toml if missing.
# (In a local Jupyter kernel you can also run: %pip install -q -e ..)
import contextlib
import os
import subprocess
import sys
from pathlib import Path

REPO_URL = "https://github.com/diaeshantony-usd/investment-research-agent"
IN_COLAB = "google.colab" in sys.modules

if IN_COLAB:
    project_root = Path("/content/investment-research-agent")
    if not project_root.exists():
        subprocess.run(["git", "clone", "--depth", "1", REPO_URL, str(project_root)
    subprocess.run(
        [sys.executable, "-m", "pip", "install", "-q", "-e", str(project_root)], ch
    )
    # Optional: API keys stored as Colab secrets override the repository defaults.
    try:
        from google.colab import userdata

        for key in ("OLLAMA_API_KEY", "NEWS_API_KEY", "FRED_API_KEY", "ALPHA_VANTAG
            # A secret that is not set keeps the repository default.
            with contextlib.suppress(Exception):
                os.environ[key] = userdata.get(key)
    except ImportError:
        pass
else:
    project_root = next(
        (p for p in (Path.cwd(), *Path.cwd().parents) if (p / "src" / "research_age
        Path.cwd(),
    )
    # Automatically install dependencies from pyproject.toml if missing in the loca
    try:
        import alpha_vantage  # noqa: F401
        import fredapi  # noqa: F401
        import matplotlib  # noqa: F401
        import newsapi  # noqa: F401
        import yfinance  # noqa: F401
    except ImportError:
        print("Installing project dependencies from pyproject.toml...")
        subprocess.run(
            [sys.executable, "-m", "pip", "install", "-q", "-e", str(project_root)]
            check=True,
        )

os.chdir(project_root)
sys.path.insert(0, str(project_root / "src"))

from research_agent.agent_framework import END, Agent, GraphMemory, GraphState, Too
from research_agent.config import DEFAULT_MODEL
from research_agent.interfaces import get_llm
```

```
# Start the demo from an empty long-term memory, so section 10 shows learning from
RESET_LONG_TERM_MEMORY = True
from research_agent.memory import LongTermMemory

long_term = LongTermMemory()
if RESET_LONG_TERM_MEMORY:
    long_term.reset()

llm = get_llm()  # shared LLM client (Ollama Cloud)
rel_project_root = project_root.name
try:
    rel_memory = Path(long_term.directory).relative_to(project_root).as_posix()
except ValueError:
    rel_memory = Path(long_term.directory).name

print(f"Environment      : {'Google Colab' if IN_COLAB else 'local'}")
print(f"Project root     : {rel_project_root}")
print(f"LLM model        : {DEFAULT_MODEL}")
print(f"Long-term memory : {rel_memory} (reset: {RESET_LONG_TERM_MEMORY})")
```

```
Environment      : local
Project root     : investment-research-agent
LLM model        : gpt-oss:120b
Long-term memory : data/memory/long_term (reset: True)
```

In [2]:

```
# Helpers used throughout the notebook.
import json
import logging
import re
import textwrap
import time

import pandas as pd
from IPython.display import Markdown, display

from research_agent.memory import ResearchSession

pd.set_option("display.max_colwidth", 90)
# yfinance logs transient download retries as errors; the tools already report miss
logging.getLogger("yfinance").setLevel(logging.CRITICAL)


def show_md(text: str) -> None:
    """Render Markdown output cleanly without MathJax currency collisions.

    Escapes bare dollar signs outside code blocks so Jupyter/MathJax does not
    treat currency figures ($150bn, $130) as LaTeX math delimiters, which strips
    whitespace and causes words to overlap in the rendered output.
    """
    parts = re.split(r"(`+[^`]*`+)", str(text))
    escaped = [
        p if p.startswith("`") else re.sub(r"(?<!\\)\$", r"\$", p)
        for p in parts
    ]
    display(Markdown("".join(escaped)))
```

```
def short(value, width: int = 110) -> str:
    """One-line preview of any value."""
    text = value if isinstance(value, str) else json.dumps(value, default=str)
    return textwrap.shorten(" ".join(text.split()), width=width, placeholder=" ..."


def ask_researched(make_session, question: str, attempts: int = 3, **ask_kwargs):
    """Ask until the Critic has reviewed a brief; returns the session.

    The Planner is an LLM and sometimes answers in prose instead of dispatching the
    (the graph then ends and ``state.review`` stays empty). A retry in a new conver
    almost always dispatches.
    """
    for attempt in range(1, attempts + 1):
        session = make_session(attempt)
        session.ask(question, show=False, **ask_kwargs)
        review = getattr(session.last_state, "review", None)
        if review and review.get("history"):
            return session
        print(f"Attempt {attempt}/{attempts}: the Planner did not dispatch research
    raise RuntimeError("No research brief was produced; run the cell again")


def round_stats(state) -> dict:
    """First-draft and released Critic scores from a run's review history."""
    rounds = state.review["history"]
    best = max(rounds, key=lambda r: (-r["blockers"], r["weighted"], r["round"]))
    return {
        "scores": [r["weighted"] for r in rounds],
        "unverified": [len(r["unverified"]) for r in rounds],
        "first_draft": rounds[0]["weighted"],
        "first_unverified": len(rounds[0]["unverified"]),
        "released": best["weighted"],
        "released_round": best["round"],
        "passed": best["passed"],
        "refinements": len(rounds) - 1,
    }
```

Part I. Agent design and workflows

1. Problem and design goals

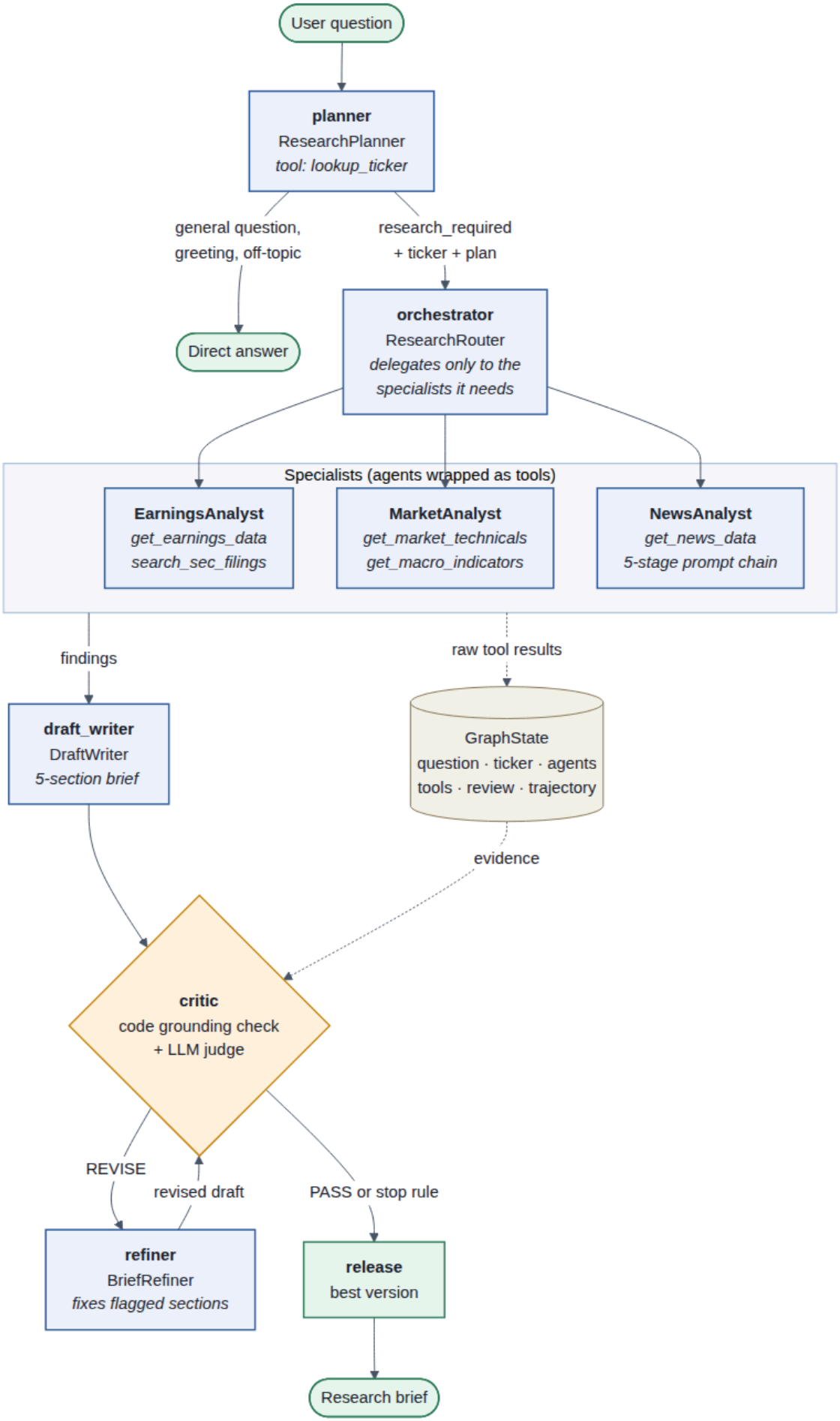
Given a company name or ticker, the agent produces a **research brief** with five sections (Executive Summary, Fundamentals, Technicals, Catalysts & Risks, Stance) in which every figure comes from live data, and it answers follow-up questions in the same conversation.

A single LLM prompt cannot do this reliably: it quotes figures from its training data, which are out of date or invented, and nothing checks the result. Each design decision below answers one of these problems.

What the brief needs	Design decision	Where in the code
Current, real figures	Agents never quote numbers from memory. Specialists fetch them with tools, and the raw results stay on the shared state (`GraphState.tools`)	`tools/` , `agent_framework.py`
Different kinds of evidence	Three specialists (earnings, market, news), each with only its own tools. A router decides which ones a question needs	`agents/`
Mistakes caught before release	A Critic checks every figure against the raw tool data in code, an LLM judge scores the rest, and a Refiner rewrites the flagged sections	`workflows/review_loop.py`
Follow-ups without re-running research	Conversation memory caches each brief per thread	`memory/session.py`
Better results over time	Long-term memory keeps per-ticker notes and recurring Critic lessons	`memory/long_term.py`
Readable, auditable runs	Every node's input, tool calls and output are recorded in `state.trajectory`	`agent_framework.py`

2. System architecture

The system is a graph of six nodes built with our own `StateGraph` class (`src/research_agent/agent_framework.py` , compiled in `workflows/graph.py`). Nodes share one `GraphState` object: each node reads what it needs from it and writes its result back.

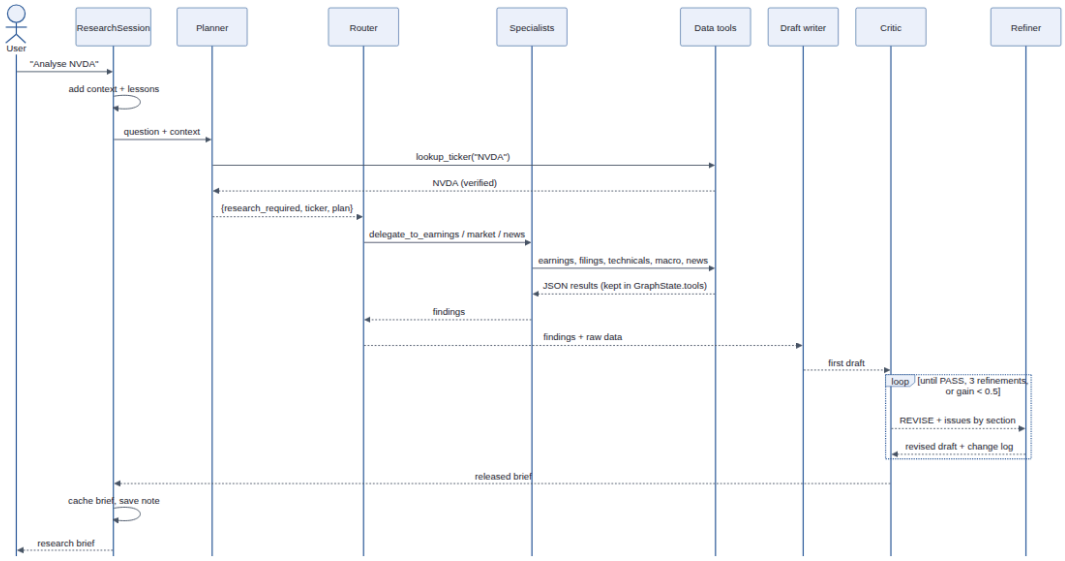

User question
planner
ResearchPlanner
tool: lookup_ticker
general question, greeting, off-topic
research_required + ticker + plan
Direct answer
orchestrator
ResearchRouter
delegates only to the specialists it needs
Specialists (agents wrapped as tools)
EarningsAnalyst
get_earnings_data
search_sec_filings
MarketAnalyst
get_market_technicals
get_macro_indicators
NewsAnalyst
get_news_data
5-stage prompt chain
findings
raw tool results
draft_writer
DraftWriter
5-section brief
GraphState
question · ticker · agents
tools · review · trajectory
evidence
critic
code grounding check
+ LLM judge
REVISE
revised draft
PASS or stop rule
refiner
BriefRefiner
fixes flagged sections
release
best version
Research brief

Node	Implementation	LLM	Tools	Reads `Graph`
`planner`	`ResearchPlanner` (`agents/planner.py`)	yes	`lookup_ticker`, checkpoint look-up tools	questi conve contex term r
`orchestrator`	`ResearchRouter` (`agents/router.py`)	yes	one `delegate_to_<specialist>` tool per specialist	planne dispat
specialists	`EarningsAnalyst`, `MarketAnalyst`, `NewsAnalyst`	yes	see section 3	task fr router
`draft_writer`	`DraftWriter` (`agents/draft_writer.py`)	yes	none	findin tool re Critic l
`critic`	`Critic` (`workflows/review_loop.py`)	code check + LLM judge	none	draft, results
`refiner`	`BriefRefiner` (`agents/refiner.py`)	yes	the five data tools	draft, issues
`release`	`release_node` (`workflows/review_loop.py`)	no	none	review

How a request flows

1. The **planner** verifies the company with `lookup_ticker` and decides whether research is needed. General finance questions, greetings and off-topic requests end here with a direct answer.
2. The **orchestrator** delegates to the specialists the question needs (all three for "Analyse NVDA"). Each specialist is an agent wrapped as a tool, so delegation is an ordinary tool call.
3. The **draft writer** combines the findings into the five-section brief.
4. The **critic** reviews it. If it does not pass, the **refiner** fixes the flagged sections and the critic reviews again. The loop stops on a pass, after 3 refinements, or when a round improves the score by less than 0.5.
5. **release** returns the best version.

The same request as a sequence of calls between components:



The cell below compiles the graph from `workflows/graph.py`; its node names are the ones in the table and the diagrams.

In [3]:
```python
from research_agent.workflows.graph import build_research_workflow

# Compile the research graph; the node names match the table above.
full_memory = GraphMemory()
workflow = build_research_workflow(llm=llm, memory=full_memory)

print("Institutional Multi-Agent Workflow compiled successfully.")
print(f"Registered Graph Nodes: {list(workflow.graph.nodes.keys())}")
```

```
Institutional Multi-Agent Workflow compiled successfully.
Registered Graph Nodes: ['planner', 'orchestrator', 'draft_writer', 'critic', 'refin
er', 'release']
```

3. Data sources and tools

Each tool is a plain Python function wrapped in our `Tool` class, which turns its signature and docstring into the JSON schema the LLM sees. Every tool returns JSON text, caches its responses on disk, and reports a failed source in the result instead of raising, so an agent can carry on with partial data.

Tool	Data source	Fallback	Used by	Returns
`lookup_ticker`	Yahoo Finance search (US listings only)	none	Planner	verified ticker and company name
`get_earnings_data`	Yahoo Finance financial statements (`yfinance`)	Alpha Vantage	EarningsAnalyst, Refiner	latest quarter revenue, margins, growth, forward P/E,

Tool	Data source	Fallback	Used by	Returns
				free cash flow, EPS history
`search_sec_filings`	SEC EDGAR, latest 10-K / 10-Q	keyword ranking if embeddings are unavailable	EarningsAnalyst, Refiner	the filing passages most relevant to a query (retrieval over embedded chunks)
`get_market_technicals`	Yahoo Finance price history	Alpha Vantage	MarketAnalyst, Refiner	price, 50/200-day SMA, 14-day RSI, beta, trend
`get_macro_indicators`	FRED (Federal Reserve Economic Data)	none	MarketAnalyst, Refiner	Fed funds rate, 10-year Treasury, CPI, each with its date
`get_news_data`	NewsAPI.org	Yahoo Finance news	NewsAnalyst, Refiner	recent articles for the company

The cell below calls two tools directly, the way an agent would.

In [4]:
```
# Two data tools called directly (each returns JSON text).
# A Tool wraps a plain Python function with a name and description for the LLM.
# Agents call tool.execute(...) (always returns text); here we call the function it

from research_agent.tools.filings import search_sec_filings
from research_agent.tools.yahoo_finance import lookup_ticker

# 1. Symbol Resolution Tool
ticker_info = lookup_ticker.func("Apple")
print("1. Tool Execution: lookup_ticker('Apple')")
print(f"   Result: {ticker_info}")

# 2. Regulatory Filing Retrieval Tool (returns JSON text with the matching passages
filing = json.loads(search_sec_filings.func("NVDA", query="risk factors competition
passage = (filing.get("passages") or [{}])[0].get("text", filing.get("error", ""))
print("\n2. Tool Execution: search_sec_filings('NVDA', query='risk factors competit
print(f"   Filing           : {filing.get('form_type')} filed {filing.get('filing_d
print(f"   Excerpt Preview  : {' '.join(passage.split())[:220]}...")
```

```
1. Tool Execution: lookup_ticker('Apple')
   Result: {'ticker': 'AAPL', 'company': 'Apple', 'source': 'yahoo_finance'}
2. Tool Execution: search_sec_filings('NVDA', query='risk factors competition')
   Filing           : 10-K filed 2026-02-25
   Excerpt Preview  : orm 10-K. Item 1A. Risk Factors The following risk factors sho
uld be considered in addition to the other information in this Annual Report on Form
10-K. The following risks could harm our business, financial condition, r...
```

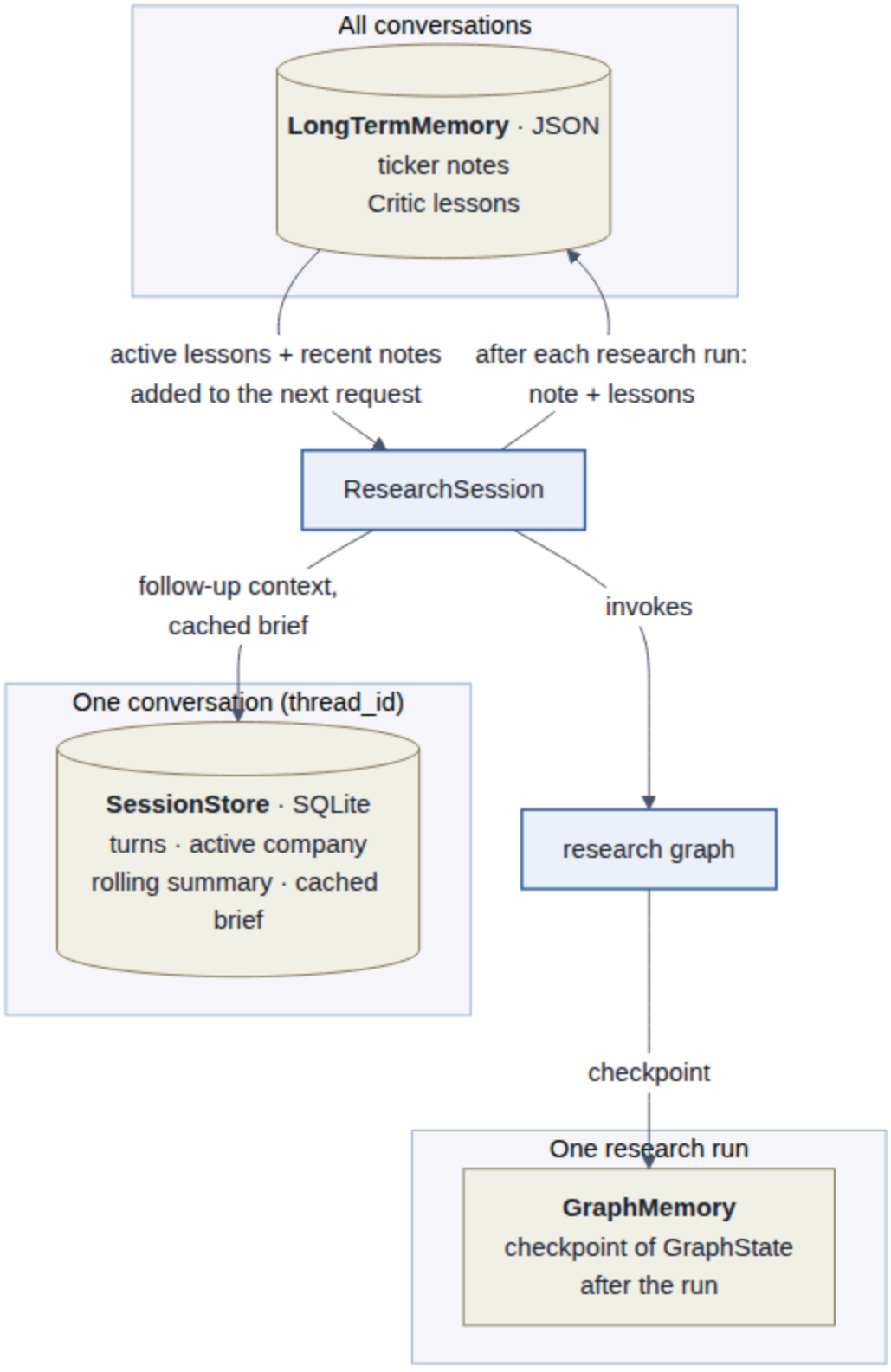
4. Framework internals (`agent_framework.py`)

The team wrote its own small agent framework instead of using LangChain or LangGraph, so every step of a run can be inspected. It has four building blocks.

- **`Tool`** wraps a Python function with a name, a description and a JSON schema. Calling a tool never raises: an error comes back as text the agent can read and react to.
- **`Agent`** has a role (its system prompt), a set of tools and an LLM. `run()` loops: the LLM either calls tools, whose results are appended to the conversation, or gives the final answer. The loop is capped by `max_iterations`. Every step is kept in `response.trace`. **`OrchestratorAgent`** turns other agents into tools (`agent.as_tool()`), which is how the router delegates to specialists.
- **`GraphState`** is the shared state passed through the graph: `question`, `active_ticker`, `agents` (each agent's output), `tools` (raw tool results, kept for the Critic), `review`, `trajectory` and `output`.
- **`StateGraph`** runs nodes (agents or Python functions) along fixed edges and conditional edges (`add_conditional_edges(node, decider=..., route_map=...)`). The two decisions in this system are `route_planner` (research or direct answer) and `route_review` (refine or release).

The system keeps three kinds of memory, each with a different lifetime:

Memory	Class (file)	Lifetime	Holds	Used for
Run checkpoints	`GraphMemory` (`agent_framework.py`)	one run	a snapshot of `GraphState` after each run	auditing a run; the Planner can look up earlier checkpoints
Conversation memory	`SessionStore` + `ResearchSession` (`memory/`)	one conversation (`thread_id`), stored in SQLite	turns, active company, rolling summary, cached brief per ticker	follow-up questions (section 11)
Long-term memory	`LongTermMemory` (`memory/long_term.py`)	across all conversations	dated notes per ticker, recurring Critic lessons	learning across runs (section 10, F4)



The three cells below show an agent with one tool, the shared state and a checkpoint.

In [5]:
```
# An Agent with one Tool: the LLM decides to call the tool, reads the result, then
def calculate_financial_health_score(operating_margin: float, debt_to_equity: float
    """Calculates a composite financial health score from operating margin and leve
    score = (operating_margin * 0.7) - (debt_to_equity * 20.0)
```

```
    verdict = "Strong" if score > 30 else "Moderate" if score > 10 else "High Risk"
    return {"composite_score": round(score, 2), "health_verdict": verdict}


health_tool = Tool(
    name="calculate_financial_health_score",
    description="Calculate composite financial health score from margin and debt ra
    func=calculate_financial_health_score,
)

analyst_agent = Agent(
    name="FinancialHealthSpecialist",
    role="Evaluate company balance sheet resilience using the health score tool.",
    llm=llm,
    tools=[health_tool],
    max_iterations=4,
)

query = "Evaluate NVDA financial health if operating margin is 62.0% and debt-to-eq
response = analyst_agent.run(query)

print(f"Agent Name     : {analyst_agent.name}")
print(f"Iterations Run : {response.iterations}")
print(f"Tools Used     : {[t.name for t in analyst_agent.tools.values()]}")
print(f"Trace Steps    : {[step.step_type for step in response.trace]}")
print(f"\n--- AGENT FINAL ANSWER ---\n{response.content}")
```

```
Agent Name     : FinancialHealthSpecialist
Iterations Run : 2
Tools Used     : ['calculate_financial_health_score']
Trace Steps    : ['tool_call', 'tool_result', 'thought', 'final_answer']

--- AGENT FINAL ANSWER ---
**NVDA – Financial Health Evaluation**

| Metric | Value |
|--------|-------|
| **Operating Margin** | **62.0 %** |
| **Debt-to-Equity Ratio** | **0.42** |

**Composite Health Score (tool-derived)**
- **Score:** **35.0** (out of a typical 0–100 scale)
- **Verdict:** **Strong**

### Interpretation
- **Operating Margin (62 %)** – Extremely high; indicates NVDA generates a large pro
fit on each dollar of revenue, reflecting efficient operations and strong pricing po
wer.
- **Debt-to-Equity (0.42)** – Low-to-moderate leverage; the company relies more on e
quity than debt, reducing financial risk and interest-expense pressure.
- **Composite Score (35 / 100, “Strong”)** – The scoring model classifies this combi
nation of high profitability and modest leverage as a strong financial position, sug
gesting resilience to economic downturns and capacity to fund growth initiatives.

### Overall Assessment
NVDA’s financial health appears robust. The high operating margin provides ample cas
h flow, while the modest debt-to-equity ratio limits risk. This combination supports
a “Strong” rating in the composite health model, indicating the company is well-posi
tioned to sustain operations, meet obligations, and invest in future growth.
```

In [6]:
```python
# GraphState: agents write their findings and raw tool results to one shared object
demo_state = GraphState(question="Analyze NVIDIA Q2 fundamentals and margins")
demo_state.active_ticker = "NVDA"

# EarningsAnalyst posts findings and raw tool observations to blackboard
demo_state.agents["EarningsAnalyst"] = "Q2 Revenue: $30.04B (+122% YoY), Gross Marg
demo_state.tools["get_earnings_data"] = {
    "revenue_b": 30.04,
    "gross_margin_pct": 75.1,
    "ebitda_b": 18.64,
}

# MarketAnalyst posts technical metrics to blackboard
demo_state.agents["MarketAnalyst"] = (
    "Current Price: $128.50, 50-day SMA: $121.30, 14-day RSI: 63.34."
)
demo_state.tools["get_market_technicals"] = {"price": 128.50, "sma_50": 121.30, "rs

print(f"Active Ticker        : {demo_state.active_ticker}")
print(f"Agents on Blackboard : {list(demo_state.agents.keys())}")
print(f"Tools Preserved      : {list(demo_state.tools.keys())}")
print(f"Blackboard Schema    : {list(demo_state.to_dict().keys())}")
```

```
Active Ticker        : NVDA
Agents on Blackboard : ['EarningsAnalyst', 'MarketAnalyst']
Tools Preserved      : ['get_earnings_data', 'get_market_technicals']
Blackboard Schema    : ['question', 'output', 'state_unique_id', 'thread_id', 'agent
s', 'tools', 'trajectory', 'refine_count', 'review']
```

In [7]:
```python
# GraphMemory: checkpoint the state and restore it by id.
import json

memory = GraphMemory()
checkpoint_id = memory.write(demo_state)

print(f"Checkpoint ID Generated: {checkpoint_id}")
print(f"Memory Checkpoints     : {memory.list_state_ids()}")

# Restore the checkpoint (stored as JSON) from memory
restored = json.loads(memory.get_state(checkpoint_id))
print(f"Restored Question      : {restored['question']}")
print(f"Restored Finding       : {restored['agents']['EarningsAnalyst']}")
```

```
Checkpoint ID Generated: state_20261018_110029_c78598
Memory Checkpoints     : [
  "state_20261018_110029_c78598"
]
Restored Question      : Analyze NVIDIA Q2 fundamentals and margins
Restored Finding       : Q2 Revenue: $30.04B (+122% YoY), Gross Margin: 75.1%.
```

5. The agent team, one component at a time

Each cell below runs one component on its own, with its input and output contract in the header comment. Section 6 then runs them together.

Component	Purpose	Tools	Input	Output
ResearchPlanner	verify the company, decide whether to research, plan the steps	`lookup_ticker`	user question	`{"research_required "ticker", "plan"}`
ResearchRouter	delegate to the specialists the question needs	`delegate_to_*`	question and plan	specialist findings
EarningsAnalyst	fundamentals, margins, valuation, filing passages	`get_earnings_data` , `search_sec_filings`	task	findings with figures fror the tools
MarketAnalyst	price trend, momentum,	`get_market_technicals` , `get_macro_indicators`	task	technical and macro findings

Component	Purpose	Tools	Input	Output
	macro backdrop			
NewsAnalyst	recent catalysts and sentiment (five-stage chain, section 12)	`get_news_data`	task	cited news digest
DraftWriter	write the five-section brief	none	findings and raw tool results	draft brief
Critic	check figures in code, score the brief with an LLM judge	none	draft and raw tool results	scores, issues, PASS / REVISE
BriefRefiner	fix only the flagged sections	the five data tools	draft and issues	revised draft and change log

5.1 ResearchPlanner

A general question should come back without research; a company question should call `lookup_ticker` and return a research dispatch.

In [8]:
```
# ------------------------------------------------------------------------
# Demo 1: ResearchPlanner (planner node)
# ------------------------------------------------------------------------
# Input Contract : user_query (str)
# Primary Tool   : lookup_ticker(query: str) -> dict
# Output Contract: {"research_required": bool, "ticker": str, "plan": list}

from research_agent.agents.planner import planner_agent

planner = planner_agent(llm=llm)

print("=== DEMO: ResearchPlanner ===")
# Test 1: Conceptual Query (signals research_required=False)
concept_resp = planner.run("What is the difference between operating margin and gro
print("\n[Query 1: Conceptual Definition]")
print(f"Output Preview: {str(concept_resp)[:160]}...")

# Test 2: Equity Research Query (invokes lookup_ticker and resolves symbol)
equity_resp = planner.run("Analyze NVIDIA financial growth and competitive position
print("\n[Query 2: Equity Research]")
print(f"Output Preview: {str(equity_resp)[:200]}...")
```

```
=== DEMO: ResearchPlanner ===
```

```
[Query 1: Conceptual Definition]
Output Preview: Operating margin and gross margin are both profitability ratios that
measure how efficiently a company turns revenue into profit, but they focus on diffe
rent st...
[Query 2: Equity Research]
Output Preview: {
  "research_required": true,
  "ticker": "NVDA",
  "plan": [
    "Gather and analyze NVIDIA's latest audited financial statements (10\u2011K, 10
\u2011Q) focusing on revenue growth, YoY and QoQ trend...
```

5.2 EarningsAnalyst

First the earnings tool is called directly to show its output, then the agent analyses NVDA using it.

In [9]:
```python
# ------------------------------------------------------------------------
# Demo 2: EarningsAnalyst (Specialist 1)
# ------------------------------------------------------------------------
# Input Contract : task_description (str)
# Primary Tools  :
#   1. get_earnings_data(ticker: str) -> dict
#      Input : ticker="NVDA"
#      Output: {"revenue": "$96.22B", "gross_margin_pct": 74.98, "forward_pe": 14.2
#   2. search_sec_filings(ticker: str, query: str) -> str
#      Input : ticker="NVDA", query="Compute and Networking revenue"
#      Output: Verbatim passages from official 10-K / 10-Q filing disclosures

from research_agent.agents.earnings_analyst import earnings_analyst_agent
from research_agent.tools.earnings import get_earnings_data

earnings_agent = earnings_analyst_agent(llm=llm)

print("=== DEMO: EarningsAnalyst & Tools ===")
# Tool Call Contract
raw_earnings = get_earnings_data.func("NVDA")
print("\n[Direct Tool Call: get_earnings_data('NVDA')]")
print(f"Quarter Ended     : {raw_earnings.get('quarter_end')}")
print(f"Revenue (quarter) : {raw_earnings.get('revenue')}")
print(f"Gross Margin      : {raw_earnings.get('gross_margin_pct', 0):.2f}%")
print(f"Operating Margin  : {raw_earnings.get('operating_margin_pct', 0):.2f}%")
print(f"Forward P/E Ratio : {raw_earnings.get('forward_pe', 0):.2f}x")

# Agent Execution Demo
agent_resp = earnings_agent.run("Provide fundamental earnings and margin analysis f
print("\n[EarningsAnalyst Agent Output Preview]")
print(str(agent_resp)[:250] + "...")
```

```
=== DEMO: EarningsAnalyst & Tools ===
```

```
[Direct Tool Call: get_earnings_data('NVDA')]
Quarter Ended     : 2026-07-31
Revenue (quarter) : $96.22B
Gross Margin      : 74.98%
Operating Margin  : 66.24%
Forward P/E Ratio : 13.45x
[EarningsAnalyst Agent Output Preview]
**NVDA – FY2026 Q3 (Quarter ended July 31 2026) – Key Fundamentals**

| Metric | Value (as reported) | Source |
|--------|---------------------|--------|
| Revenue (Quarter) | **$96.22 B** | `get_earnings_data` |
| Gross Margin | **74.98 %** | `get_ea...
```

5.3 MarketAnalyst

Price trend and momentum from Yahoo Finance, and the macro backdrop from FRED.

In [10]:

```python
# ------------------------------------------------------------------------
# Demo 3: MarketAnalyst (Specialist 2)
# ------------------------------------------------------------------------
# Input Contract : task_description (str)
# Primary Tools  :
#   1. get_market_technicals(ticker: str) -> dict
#      Input : ticker="NVDA"
#      Output: {'current_price': 226.39, '50_day_sma': 222.46, 'rsi_14': 49.53, ...
#   2. get_macro_indicators() -> dict
#      Input : None
#      Output: {"fed_funds_rate": {"value": 3.75, "date": ...}, "us_10y_treasury":

import json

from research_agent.agents.market_analyst import market_analyst_agent
from research_agent.tools.macro import get_macro_indicators
from research_agent.tools.market import get_market_technicals

market_agent = market_analyst_agent(llm=llm)

print("=== DEMO: MarketAnalyst & Tools ===")
# Tool Call Contracts
tech_data = json.loads(get_market_technicals.func("NVDA"))  # tools return JSON tex
macro_data = json.loads(get_macro_indicators.func())

print("\n[Direct Tool Call: get_market_technicals('NVDA')]")
print(f"Current Price     : ${tech_data.get('current_price', 0):.2f}")
print(f"50-Day SMA        : ${tech_data.get('50_day_sma', 0):.2f}")
print(f"200-Day SMA       : ${tech_data.get('200_day_sma', 0):.2f}")
print(f"14-Day RSI        : {tech_data.get('rsi_14', 0):.2f}")
print(f"Technical Trend   : {tech_data.get('trend', 'Neutral')}")

print("\n[Direct Tool Call: get_macro_indicators()]")
for key, label in [
    ("fed_funds_rate", "Fed Funds Rate"),
    ("us_10y_treasury", "10-Yr Treasury"),
    ("cpi_yoy", "CPI Inflation YoY"),
```

```
]:
    entry = macro_data.get(key) or {}
    print(f"{label:<18}: {entry.get('value', 'n/a')}% (as of {entry.get('date', '-'
```

```
=== DEMO: MarketAnalyst & Tools ===
[Direct Tool Call: get_market_technicals('NVDA')]
Current Price     : $214.13
50-Day SMA        : $222.96
200-Day SMA       : $202.66
14-Day RSI        : 37.56
Technical Trend   : Moderate Uptrend

[Direct Tool Call: get_macro_indicators()]
Fed Funds Rate    : 3.75% (as of 2026-09-01)
10-Yr Treasury    : 5.1% (as of 2026-10-15)
CPI Inflation YoY : 4.02% (as of 2026-09-01)
```

5.4 NewsAnalyst

The news tool's raw articles, then the analyst's cited digest (its five stages are shown in section 12).

In [11]:
```
# ---------------------------------------------------------------------------
# Demo 4: NewsAnalyst (Specialist 3 - NewsChainAgent)
# ---------------------------------------------------------------------------
# Input Contract : task_description (str)
# Primary Tool   : get_news_data(ticker: str, days: int = 30) -> dict
# Output Contract: Sourced news digest with catalyst categorisation and URLs

import json

from research_agent.agents.news_analyst import news_analyst_agent
from research_agent.tools.news import get_news_data

news_agent = news_analyst_agent(llm=llm)

print("=== DEMO: NewsAnalyst (5-Stage Pipeline) ===")
news_data = json.loads(get_news_data.func("NVDA", days=30))  # JSON text -> dict
print(f"Articles Retrieved: {len(news_data.get('articles', []))}")
if news_data.get("articles"):
    first = news_data["articles"][0]
    print(f"Top Headline      : {first.get('headline')}")
    print(f"Source URL        : {first.get('url')}")
    print(f"Publication Date  : {first.get('date')}")
```

```
=== DEMO: NewsAnalyst (5-Stage Pipeline) ===
Articles Retrieved: 9
Top Headline      : Nvidia says its new AI safety platform can contain rogue agents
within ‘milliseconds’
Source URL        : https://www.theverge.com/tech/1001287/nvidia-ai-safety-platform-
rogue-agents
Publication Date  : 2026-09-28
```

5.5 Critic and review routing

A small draft whose figures all match the tool data: the Critic should verify them and the review router should choose the next step.

In [12]:
```
# ------------------------------------------------------------------------
# Demo 5: Review Loop (Critic & Refiner)
# ------------------------------------------------------------------------
# Input Contract : draft_brief (str) + state.tools (dict)
# Primary Nodes  :
#   1. critic_node : Verifies numbers against raw tools -> scores dimensions -> sta
#   2. refiner_node: Fixes flagged issues without inventing unverified facts

from research_agent.agents import refiner_agent
from research_agent.workflows.review_loop import make_review_nodes

critic_fn, refiner_fn, release_fn, route_fn = make_review_nodes(
    llm=llm, refiner=refiner_agent(llm=llm)
)

test_state = GraphState(question="Audit NVDA Brief")
test_state.tools["NVDA"] = {
    "get_earnings_data": {"revenue": 96.22e9, "gross_margin": 74.98},
    "get_market_technicals": {"current_price": 226.39, "sma_50": 222.46},
}
test_state.agents["DraftWriter"] = (
    "# NVDA Brief\nRevenue reached $96.22B with gross margin of 74.98%. "
    "Current stock price is $226.39 above the 50-day SMA of $222.46."
)

audit_delta = critic_fn(test_state)
review_status = audit_delta.get("review_status", "")

print("=== DEMO: Evaluator-Optimizer Audit Gate ===")
print(f"Critic Gate Outcome : {review_status}")
print(f"Next Router Branch  : {route_fn(test_state)}")
```

```
=== DEMO: Evaluator-Optimizer Audit Gate ===
Critic Gate Outcome : REVISE 3.0/10, 0 blockers
Next Router Branch  : refine
```

6. End-to-end run: researching NVIDIA

The full graph runs through `ResearchSession`, the same entry point the command-line app (`cli.py`) and the Gradio app use. The session adds conversation context to the request and, after a research run, saves what the Critic found to long-term memory (section 10).

While it runs, each line shows the agent at work (`Agent: [...]`) or a tool call with its real arguments (`Tool: [...]`).

In [13]:
```
# Research NVDA end to end. A new thread_id gives every run of this notebook a fres
main_question = (
    "Provide an institutional equity research briefing on NVDA (NVIDIA Corporation)
```

```
    " covering fundamentals, valuation, technical momentum, and news catalysts."
)
def _new_main_session(attempt: int) -> ResearchSession:
    return ResearchSession(
        thread_id=f"walkthrough-nvda-{attempt}-{int(time.time())}",
        memory=full_memory,
        long_term=long_term,
        workflow=workflow,
    )



def _progress_listener(msg: str) -> None:
    print(f"  {msg}")


print(f"Question: {main_question}\n")
started = time.time()
main_session = ask_researched(_new_main_session, main_question, on_progress=_progre
main_state = main_session.last_state

print(f"\nRun time         : {time.time() - started:.0f} s")
print(f"Resolved ticker  : {main_session.active_ticker}")
print(f"Review status    : {getattr(main_state, 'review_status', 'n/a')}")
print(f"Trajectory steps : {len(main_state.trajectory)}")
```

```
Question: Provide an institutional equity research briefing on NVDA (NVIDIA Corporat
ion) covering fundamentals, valuation, technical momentum, and news catalysts.

  Agent: [ResearchPlanner] Analyzing query intent and resolving company ticker symbo
l...
  Tool: [ResearchPlanner] lookup_ticker(company_name='NVIDIA Corporation', query='NV
DA')
  Agent: [ResearchRouter] Decomposing research brief and dispatching specialist anal
yst team...
  Tool: [ResearchRouter] delegate_to_earningsanalyst(task_description='Perform a det
ailed fundamentals ...')
  Tool: [EarningsAnalyst] get_earnings_data(ticker='NVDA')
  Tool: [EarningsAnalyst] search_sec_filings(form_type='10-Q', ticker='NVDA', query
='segment performance highlights',...)
  Tool: [EarningsAnalyst] search_sec_filings(form_type='10-K', query='balance sheet
risk', ticker='NVDA', top_k=5)
  Tool: [EarningsAnalyst] search_sec_filings(form_type='10-K', query='liquidity', ti
cker='NVDA', top_k=5)
  Tool: [ResearchRouter] delegate_to_marketanalyst(task_description='Provide a techn
ical momentum and...')
  Tool: [MarketAnalyst] get_market_technicals(ticker='NVDA')
  Tool: [MarketAnalyst] get_macro_indicators(category='all')
  Tool: [ResearchRouter] delegate_to_newsanalyst(task_description='Gather the last 3
0 days of news ...')
  Tool: [NewsAnalyst] get_news_data(ticker='NVDA', days=30)
  Agent: [DraftWriter] Synthesizing specialist evidence into structured institutiona
l brief...
  Agent: [Critic] Auditing draft report for numeric accuracy and factual groundin
g...
  Agent: [Refiner] Refining draft sections and grounding unverified assertions...
```

```
  Agent: [Critic] Auditing draft report for numeric accuracy and factual groundin
g...
  Agent: [Release] Finalizing published institutional research briefing...

Run time         : 76 s
Resolved ticker  : NVDA
Review status    : REVISE 8.8/10, 1 blockers
Trajectory steps : 7
```

Execution trajectory

Every node appends a step to `state.trajectory`: which node ran, which tools it called with which arguments, and what it decided. The table is the audit trail of the run above.

In [14]:
```python
# One row per node execution: what ran, which tools it called, and the outcome.
def _tool_calls(step: dict) -> str:
    calls = [a.get("tool", "") for a in step.get("activities", []) if a.get("tool")
    for agent, tools in (step.get("delegations") or {}).items():
        calls += [f"{agent}.{tool}" for tool in tools]
    return ", ".join(dict.fromkeys(calls)) or "-"


trajectory_table = pd.DataFrame(
    [
        {
            "Step": step["step"],
            "Node": step["node"],
            "Executed by": step.get("executor", "system"),
            "Tools called": short(_tool_calls(step), 90),
            "Outcome": short(step.get("summary", ""), 90),
        }
        for step in main_state.trajectory
    ]
)
trajectory_table
```

Out[14]:

	Step	Node	Executed by	Tools called	Outcome
0	1	planner	ResearchPlanner	lookup_ticker	{"research_required": true, "ticker": "NVDA", "plan": ["Gather and analyze the l...
1	2	orchestrator	ResearchRouter	delegate_to_earningsanalyst, delegate_to_marketanalyst, delegate_to_newsanalyst, ...	**NVIDIA Corporation (NVDA) – Institutional Research Briefing** *Date: 18 Oct 20...
2	3	draft_writer	DraftWriter	-	# Research Brief: NVDA **Date:** 18 Oct 2026 --- ## Executive Summary NVIDIA rem...
3	4	critic	critic_node	-	State updated: review_status=REVISE 8.7/10, 1 blockers
4	5	refiner	refiner_node	-	State updated: refine_summary=round 1: 5 changes
5	6	critic	critic_node	-	State updated: review_status=REVISE 8.8/10, 1 blockers
6	7	release	release_node	-	State updated: released=round 1 (with notes)

Released research brief

The brief below is the version `release` returned. Every figure in it was checked against the raw tool results by the Critic (section 9).

In [15]:
```
# The released brief, rendered as Markdown.
show_md(main_state.output)
```

Research Brief: NVDA

Date: 18 Oct 2026

Executive Summary

NVIDIA remains the dominant platform for AI-accelerated compute, delivering **>100 % YoY revenue growth** with **ultra-high margins** (≈75 % gross, ≈66 % operating). The company generates **free-cash-flow that exceeds its revenue** and has added a **$150 bn share-buyback tranche**, reinforcing EPS support. Technicals show the stock modestly discounted to its 50-day SMA while still above the 200-day SMA, indicating a **moderate up-trend**.

High-level stance: Bullish – the combination of explosive top-line growth, premium profitability, strong cash generation, and a supportive technical backdrop outweighs the near-term supply-chain and regulatory headwinds.

---

1. Fundamental & Earnings Performance

Metric (Q2 2026)	Figure
Revenue	**$96.22 B**
Gross margin	**74.98 %**
Operating margin	**66.24 %**
Net margin	**62.03 %**
Annualized free-cash-flow	**$96.68 B**
Revenue QoQ growth	**+17.9 %**
Revenue YoY growth	**+105.85 %**
Forward P/E	**13.45×**
EPS (quarter)	**$2.22** (≈6.2 % beat vs. $2.091)
Market-cap	**$5.17 T**

Segment breakdown (10-Q, 26 Aug 2026)

Segment	Revenue	Operating Income
Compute & Networking	**$88.3 B**	**$62.7 B**
Graphics	**$7.9 B**	**$3.9 B**

Segment	Revenue	Operating Income
Total	**$96.22 B**	**$66.60 B**

Sources: `get_earnings_data` , `search_sec_filings` *(10-Q segment table).*

Valuation snapshot

- Forward P/E **≈13.5×**, modest given >100 % YoY growth and **66 % operating margin**.
- EV-based multiples (EV/EBITDA, EV/Revenue) could not be computed because the data service did not supply debt or cash figures.

Balance-sheet & risk highlights (10-K)

- **Supply-chain risk:** "Long manufacturing lead times and uncertain supply … could lead to mismatches between supply and demand."
- **Customer concentration:** Two unnamed customers represent a material share of revenues; the filing does not disclose the exact percentage.
- **Liquidity & credit risk:** Large portfolio of liquid investments but exposure to market-wide liquidity strains.
- **Third-party fab dependency:** Reliance on TSMC limits direct control over capacity.

---

2. Technical Profile & Market Momentum

Technical metric	Value
Current price	**$214.13**
50-day SMA	**$222.96**
200-day SMA	**$202.66**
14-day RSI	**37.6**
Beta (5-yr)	**2.216**
52-wk high / low	**$243.37 / $164.27**

Key derived observations

- Price is **≈4 % below the 50-day SMA**.
- Price is **≈5.7 % above the 200-day SMA**.
- RSI at 37.6 sits near the oversold boundary, suggesting limited downside pressure.
- High beta (2.22) implies amplified moves on market sentiment.

Support / resistance

- **Support:** around the 200-day SMA ($202.66).
- **Resistance:** near the 50-day SMA ($222.96).

Macro backdrop (latest): Fed Funds 3.75 %, 10-yr Treasury 5.1 %, CPI YoY 4.0 %. A higher-rate environment raises discount rates for growth stocks but does not appear to have stalled AI-driven demand.

---

3. Catalysts, Macro Context & Risk Analysis

Catalyst	Description	Expected Impact
Share-buyback expansion	Board added **$150 bn** to buyback, raising total authorisation to **$235 bn** through FY 2028.	Boosts EPS, signals confidence, provides price floor.
UK AI-safety commitment	**$250 M** of compute pledged over five years to UK research.	Enhances reputation, opens partnership pipeline.
Microsoft Surface laptop	$2,599 device powered by NVIDIA GPU, claimed double performance vs. comparable MacBook.	Drives consumer-device GPU demand, broadens market exposure.
Open Agent Safety Platform	New AI-safety software/hardware platform backed by >100 organisations.	Potential SaaS revenue, deepens ecosystem lock-in.
Smuggling allegation	Prosecutors allege **$300 M** worth of NVIDIA chips illegally shipped to China.	Legal/regulatory risk; possible fines and reputational effect.

Macro context – Strong macro demand for AI compute offsets higher interest rates; however, elevated rates increase discount rates applied to high-growth valuations.

Risk matrix

Category	Specific Risk	Potential Effect
Operational	Supply-chain constraints, reliance on TSMC	Margin pressure, production bottlenecks
Customer Concentration	Two major customers dominate revenue	Revenue volatility if a key customer cuts spend
Regulatory / Legal	Smuggling probe (≈$300 M exposure)	Legal costs, fines, negative sentiment
Macro-economic	Higher interest rates (Fed 3.75 %)	Higher discount rates may compress multiples
Valuation	EV-multiple data unavailable; reliance on forward P/E	Valuation could be less robust if debt rises

---

4. Investment Stance & Outlook

- **Fundamentals:** Revenue >$96 B with >100 % YoY growth; gross and operating margins near 75 % and 66 % respectively; free-cash-flow surpasses revenue.
- **Valuation:** Forward P/E ≈13.5× is modest relative to growth; lack of EV metrics limits full enterprise-valuation assessment.
- **Technicals:** Stock trades modestly below the 50-day SMA but comfortably above the 200-day SMA, indicating a **moderate up-trend**; RSI suggests limited further downside.
- **Catalysts:** Strong buy-back, AI-safety platform, and consumer-device partnership provide near-term upside; smuggling allegation is the primary downside risk.

Overall view: Buy – the combination of explosive top-line growth, industry-leading profitability, abundant cash generation, and a supportive technical backdrop outweighs the identifiable risks.

Target price & risk parameters (derived from technical levels):

- **Upside target:** $240 – $250 (near prior 52-week high).
- **Stop-loss:** Around the 200-day SMA (~$202.7).

All figures and narrative are taken directly from the specialist agents' outputs; no external estimates or extrapolations have been introduced.

Reviewer notes

This brief did not pass review (score 8.8/10). Open issues:

- [blocker] 2. Technical Profile & Market Momentum; 4. Investment Stance & Outlook: figures not found in the retrieved data: 2. Technical Profile & Market Momentum: '5.7 %'; 4. Investment Stance & Outlook: '$240', '$250' -> replace each with the exact figure from the tool data, write a calculated percentage as '≈N%' next to one of its inputs, or remove it
- [major] Investment Stance & Outlook: Target-price range ($240-$250) and stop-loss level are presented without any grounding in the provided data, violating the rule against extrapolated figures. -> Either remove the explicit price targets or derive them from disclosed valuation multiples (e.g., forward P/E, EV/EBITDA) using the available earnings and market-cap data, and explicitly show the calculation.
- [minor] Fundamental & Earnings Performance – Balance: The brief leans heavily toward a bullish narrative; the bearish side is limited to a list of risks without a cohesive bear case. -> Add a short paragraph that explains how the identified risks (supply-chain constraints, customer concentration, legal probe, higher rates) could translate into earnings pressure, margin compression, or share-price weakness.
- [minor] Valuation Snapshot: Enterprise-value multiples are omitted because debt and cash figures were not retrieved, leaving the valuation analysis incomplete. -> Fetch the latest balance-sheet data (total debt, cash & cash equivalents) from the 10-K filing and compute EV/EBITDA and EV/Revenue to provide a fuller valuation picture.

Part II. Agent functions and capabilities

7. F1 - Planning its research steps

The **Planner** is the first node of the graph. For a company question it:

1. verifies the company with the `lookup_ticker` tool (Yahoo Finance), and
2. returns a JSON dispatch with the verified ticker and an **ordered research plan** of 3-6 steps, which the router and specialists receive as context.

If the ticker cannot be verified, the planner asks the user instead of guessing. General questions are answered directly without research (section 13 shows this routing).

In [16]:
```
from research_agent.agents import planner_agent

planner = planner_agent(llm=llm, tools=GraphMemory().as_tools())
plan_response = planner.run("Analyse NVIDIA: is the AI data-center growth sustainab

print("Tool calls made by the planner:")
for step in plan_response.trace:
    if step.step_type == "tool_call":
        print(f"  - {step.content['tool']}({step.content['arguments']})")
    elif step.step_type == "tool_result":
        print(f"    -> {short(step.content)}")

plan = plan_response.data or {}
print("\nPlanner dispatch (JSON):")
print(json.dumps(plan, indent=2))
print("\nOrdered research plan:")
for n, step in enumerate(plan.get("plan", []), 1):
    print(f"  {n}. {step}")
```

```
Tool calls made by the planner:
  - lookup_ticker({'company_name': 'NVIDIA', 'query': 'NVIDIA'})
    -> { "ticker": "NVDA", "company": "NVIDIA Corporation", "source": "yahoo_financ
e" }

Planner dispatch (JSON):
{
  "research_required": true,
  "ticker": "NVDA",
  "plan": [
    "Collect and analyze the latest quarterly and annual financial statements, focus
ing on data\u2011center revenue trends, gross margins, and operating expenses to ass
ess underlying profitability.",
    "Review historical and forward\u2011looking data\u2011center growth rates, capac
ity utilization, and compare against AI workload demand forecasts from major cloud p
roviders and OEMs.",
    "Examine macro\u2011economic factors and industry dynamics, including AI chip su
pply chain constraints, competition, and pricing pressure, to gauge sustainability o
f growth.",
    "Identify recent and upcoming product launches, technology roadmaps, and strateg
ic partnerships that could drive or hinder future data\u2011center revenue.",
    "Aggregate and synthesize recent news, analyst commentary, and earnings call tra
nscripts for qualitative signals on management\u2019s confidence in AI data\u2011cen
ter outlook."
  ]
}

Ordered research plan:
  1. Collect and analyze the latest quarterly and annual financial statements, focus
ing on data-center revenue trends, gross margins, and operating expenses to assess u
nderlying profitability.
  2. Review historical and forward-looking data-center growth rates, capacity utiliz
ation, and compare against AI workload demand forecasts from major cloud providers a
nd OEMs.
  3. Examine macro-economic factors and industry dynamics, including AI chip supply
chain constraints, competition, and pricing pressure, to gauge sustainability of gro
wth.
  4. Identify recent and upcoming product launches, technology roadmaps, and strateg
ic partnerships that could drive or hinder future data-center revenue.
  5. Aggregate and synthesize recent news, analyst commentary, and earnings call tra
nscripts for qualitative signals on management’s confidence in AI data-center outloo
k.
```

8. F2 - Using tools dynamically

No agent follows a fixed script: each specialist's LLM chooses which of its tools to call and with which arguments (for example the filing search query), based on the question and the plan. The table lists every tool call made in the section 6 run. Its raw results stay on the state (`state.tools`), which is the evidence the Critic checks the brief against.

In [17]:
```python
# The raw tool results each agent fetched (the evidence the Critic checks against).
tool_rows = []
for agent_name, tools in main_state.tools.items():
```

```
    for tool_name, result in (tools or {}).items():
        tool_rows.append(
            {"Agent": agent_name, "Tool": tool_name, "Result (preview)": short(resu
        )
pd.DataFrame(tool_rows)
```

Out[17]:

	Agent	Tool	Result (preview)
0	ResearchPlanner	lookup_ticker	{"ticker": "NVDA", "company": "NVIDIA Corporation", "source": "yahoo_finance"}
1	ResearchRouter	delegate_to_earningsanalyst	**NVIDIA Corporation (NVDA) – Fundamentals & Valuation Snapshot** *All figures are tak...
2	ResearchRouter	delegate_to_marketanalyst	**NVIDIA Corporation (NVDA) – Technical Snapshot** | Metric | Value | |--------|------...
3	ResearchRouter	delegate_to_newsanalyst	## News digest: NVDA (last 30 days, 9 articles) **Overall news sentiment:** Bullish (8...
4	EarningsAnalyst	get_earnings_data	{"ticker": "NVDA", "quarter_end": "2026-07-31", "revenue": "$96.22B", "gross_margin_pc...
5	EarningsAnalyst	search_sec_filings	{"ticker": "NVDA", "form_type": "10-Q", "filing_date": "2026-08-26", "search_method": ...
6	EarningsAnalyst	search_sec_filings#2	{"ticker": "NVDA", "form_type": "10-K", "filing_date": "2026-02-25", "search_method": ...
7	EarningsAnalyst	search_sec_filings#3	{"ticker": "NVDA", "form_type": "10-K", "filing_date": "2026-02-25", "search_method": ...
8	MarketAnalyst	get_market_technicals	{"ticker": "NVDA", "current_price": 214.13, "52_week_high": 243.37, "52_week_low": 164...
9	MarketAnalyst	get_macro_indicators	{"fed_funds_rate": {"series_id": "FEDFUNDS", "date": "2026-09-01", "value": 3.75}, "us...
10	NewsAnalyst	get_news_data	{"ticker": "NVDA", "from": "2026-09-18", "to": "2026-10-18", "articles": [{"headline":...
11	BriefRefiner	search_sec_filings	{"ticker": "NVDA", "form_type": "10-K", "filing_date": "2026-02-25", "search_method": ...

9. F3 - Self-reflection: the Critic reviews its own output

Before anything is released, the system reviews its own draft. The **Critic** (`workflows/review_loop.py`) has three stages:

1. **Code checks (no LLM).** Every figure in the brief must match a number in the raw tool results (allowing for its rounding and units), or be a percentage calculated from figures on the same line. All five required sections must be present.
2. **LLM judge.** Scores coverage, balance, clarity and actionability from 0 to 10 and lists issues by section. The grounding score (30% of the total) comes from the code check, so the model cannot argue its way past an invented number.
3. **Verdict.** PASS needs no unverified figures, no missing sections and a weighted score of at least 8/10. Otherwise REVISE, with the issues the Refiner must fix.

Dimension	Weight	Scored by
Grounding	30%	code (share of verified figures)
Coverage	20%	LLM judge
Balance	20%	LLM judge
Clarity	15%	LLM judge
Actionability	15%	LLM judge

The table shows each review round of the section 6 run, followed by the Critic's last report and the Refiner's change log.

In [18]:
```
from research_agent.workflows.review_loop import DIMENSION_WEIGHTS

history = main_state.review["history"]
review_table = pd.DataFrame(
    [
        {
            "Round": r["round"],
            "Version": "first draft" if r["round"] == 0 else f"refinement {r['round
            **{d.capitalize(): r["scores"][d] for d in DIMENSION_WEIGHTS},
            "Weighted": r["weighted"],
            "Unverified figures": ", ".join(r["unverified"]) or "none",
            "Verdict": "PASS" if r["passed"] else "REVISE",
        }
        for r in history
    ]
)
display(review_table)

print("Critic report on the last version:")
show_md(main_state.agents["ResearchCritic"])
for n, log in enumerate(main_state.review["change_logs"], 1):
    print(f"\nRefiner change log, round {n}:")
    for item in log:
        print(f"  - {item}")
```

	Round	Version	Grounding	Coverage	Balance	Clarity	Actionability	Weighted	Unve fi
0	0	first draft	9.2	9.0	8.0	9.0	8.0	8.71	0 0 $235
1	1	refinement 1	9.5	9.0	8.0	9.0	8.0	8.80	$240

```
Critic report on the last version:
```
SCORE: 8.8/10 STATUS: REVISE (review round 1)

Dimension	Weight	Score
Grounding	30%	9.5
Coverage	20%	9.0
Balance	20%	8.0
Clarity	15%	9.0
Actionability	15%	8.0

Issues:

1. [blocker] 2. Technical Profile & Market Momentum; 4. Investment Stance & Outlook: figures not found in the retrieved data: 2. Technical Profile & Market Momentum: '5.7 %'; 4. Investment Stance & Outlook: '$240', '$250' -> replace each with the exact figure from the tool data, write a calculated percentage as '≈N%' next to one of its inputs, or remove it
2. [major] Investment Stance & Outlook: Target-price range ($240-$250) and stop-loss level are presented without any grounding in the provided data, violating the rule against extrapolated figures. -> Either remove the explicit price targets or derive them from disclosed valuation multiples (e.g., forward P/E, EV/EBITDA) using the available earnings and market-cap data, and explicitly show the calculation.
3. [minor] Fundamental & Earnings Performance – Balance: The brief leans heavily toward a bullish narrative; the bearish side is limited to a list of risks without a cohesive bear case. -> Add a short paragraph that explains how the identified risks (supply-chain constraints, customer concentration, legal probe, higher rates) could translate into earnings pressure, margin compression, or share-price weakness.
4. [minor] Valuation Snapshot: Enterprise-value multiples are omitted because debt and cash figures were not retrieved, leaving the valuation analysis incomplete. -> Fetch the latest balance-sheet data (total debt, cash & cash equivalents) from the 10-K filing and compute EV/EBITDA and EV/Revenue to provide a fuller valuation picture.

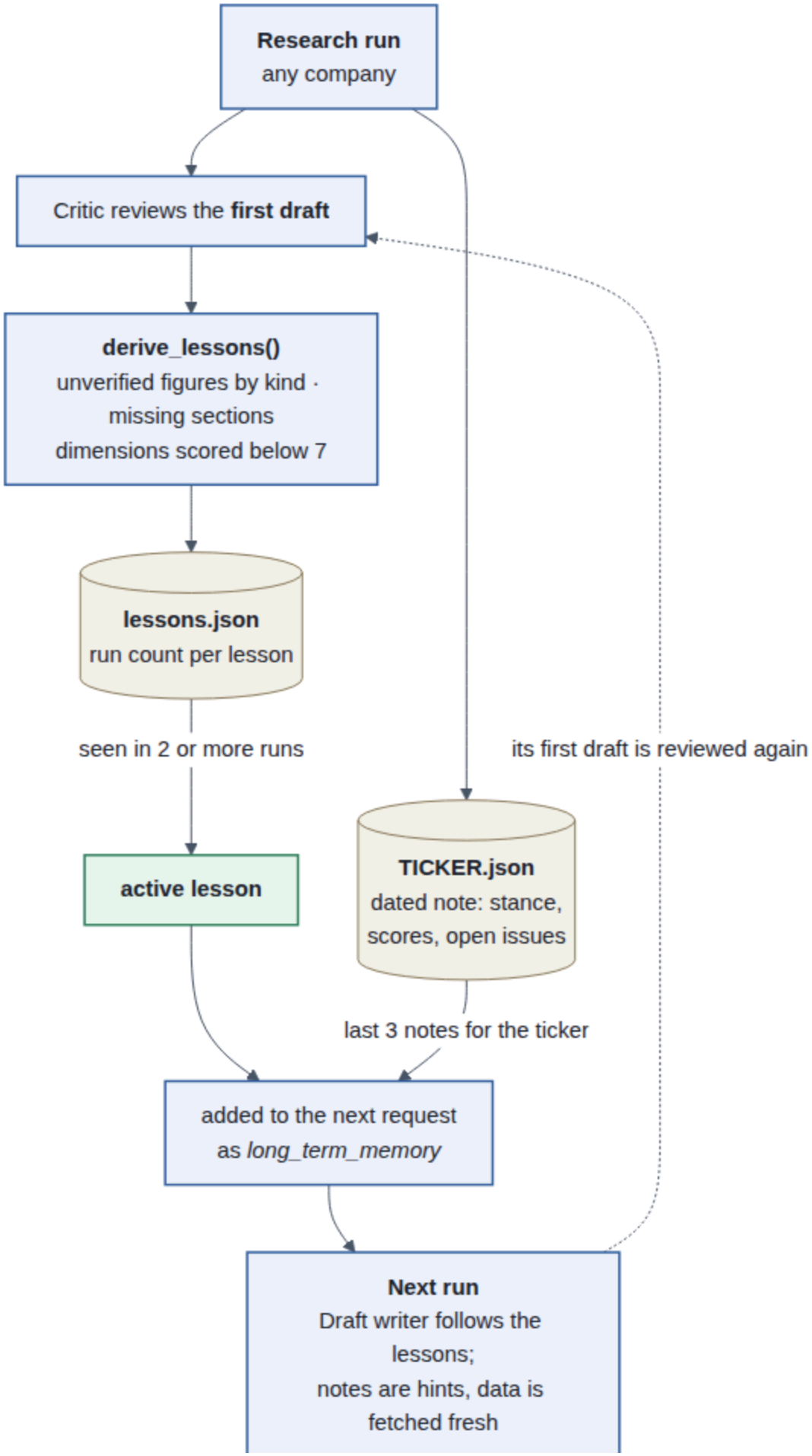
```
Refiner change log, round 1:
  - Replaced vague “>60 % operating margin” with the precise **66 % operating margin
** in the Fundamental section.
  - Removed exact decimal derivations (0.0396, 0.0566) and inserted rounded percenta
ges (“≈4 % below the 50-day SMA”, “≈5.7 % above the 200-day SMA”) in the Technical P
rofile section.
  - Updated Investment Stance target price to **$240 – $250**, citing the 52-week hi
gh as the technical resistance level, and removed reliance on buy-back amounts.
  - Confirmed the UK AI-safety commitment as **$250 M** of compute and re-phrased ac
cordingly.
  - Added note that the exact revenue share of the two largest customers is not disc
losed in the filing, maintaining transparency in the Balance-Sheet & Risk Highlight
s.
```

10. F4 - Learning across runs (long-term memory)

Long-term memory (`memory/long_term.py`) outlives conversations. After every research run, `ResearchSession` saves two things:

- **Ticker notes:** a dated note with the stance taken, the first-draft and final Critic scores and any issues left open.
- **Critic lessons:** recurring first-draft weaknesses, taken from the Critic's structured report (invented price levels, unsupported multiples, a missing section, a low balance score, ...). A lesson becomes **active** only after it has appeared in two or more runs, so one bad run cannot teach a wrong lesson.

Before the next run, the active lessons and the ticker's recent notes are added to the request. The Draft writer follows the lessons, and the notes are treated as dated hints, never as facts: data is always fetched fresh.


Research run
any company
Critic reviews the first draft
derive_lessons()
unverified figures by kind ·
missing sections
dimensions scored below 7
lessons.json
run count per lesson
seen in 2 or more runs
its first draft is reviewed again
active lesson
TICKER.json
dated note: stance,
scores, open issues
last 3 notes for the ticker
added to the next request
as long_term_memory
Next run
Draft writer follows the
lessons;
notes are hints, data is
fetched fresh

The demonstration: the NVDA run in section 6 is the first entry. Two more companies (AAPL and MSFT) are researched in new conversations, so weaknesses that recur across first drafts become active lessons. Then NVDA is researched again in a brand-new conversation, and its first draft is compared with the first one.

In [19]:
```
# ResearchSession saved the section 6 NVDA run to long-term memory automatically.
nvda_notes = long_term.notes("NVDA")
print("NVDA notes saved:", len(nvda_notes))
print(json.dumps(nvda_notes[-1], indent=2))
print(
    "Lessons seen so far:",
    len(long_term.lessons(active_only=False)),
    "| active (seen in 2+ runs):",
    len(long_term.lessons()),
)
```

```
NVDA notes saved: 1
{
  "date": "2026-10-18T05:32:22+00:00",
  "question": "Provide an institutional equity research briefing on NVDA (NVIDIA Cor
poration) covering fundamentals, valuation, technical momentum, and news catalyst
s.",
  "stance": "Bullish",
  "first_draft_score": 8.71,
  "final_score": 8.8,
  "passed": false,
  "refinements": 1,
  "first_draft_unverified": 5,
  "open_issues": [
    "2. Technical Profile & Market Momentum; 4. Investment Stance & Outlook: figures
not found in the retrieved data: 2. Technical Profile & Market Momentum: '5.7 %'",
    "Investment Stance & Outlook: Target\u2011price range ($240\u2011$250) and stop
\u2011loss level are presented without any grounding in the provided data, violating
the rule aga",
    "Fundamental & Earnings Performance \u2013 Balance: The brief leans heavily towa
rd a bullish narrative; the bearish side is limited to a list of risks without a coh
es"
  ]
}
Lessons seen so far: 3 | active (seen in 2+ runs): 0
```

10.1 Two more companies, each in a new conversation

AAPL and MSFT are researched end to end. Their first-draft weaknesses are counted alongside NVDA's.

In [20]:
```
# Two more research runs (different companies, new conversations) add to long-term
for symbol, name in [("AAPL", "Apple"), ("MSFT", "Microsoft")]:
    learn_session = ResearchSession(
        thread_id=f"walkthrough-{symbol.lower()}-{int(time.time())}", long_term=lon
    )
```

```
    learn_session.ask(f"Analyse {name} ({symbol})", show=False)
    if learn_session.last_state is not None and learn_session.last_state.review.get
        stats = round_stats(learn_session.last_state)
        print(
            f"{symbol}: first draft {stats['first_draft']:.1f}, released {stats['re
            f"{stats['refinements']} refinement(s)"
        )
    else:
        print(f"{symbol}: no reviewed brief (data unavailable)")

print(
    "\nLessons seen:",
    len(long_term.lessons(active_only=False)),
    "| active (seen in 2+ runs):",
    len(long_term.lessons()),
)
```

```
AAPL: first draft 9.5, released 9.5, 1 refinement(s)
MSFT: first draft 8.5, released 9.3, 1 refinement(s)

Lessons seen: 4 | active (seen in 2+ runs): 1
```

10.2 What the system has learned

Every lesson with the number of runs it appeared in. Lessons seen in two or more runs are active and go into the next run's context, shown below the table.

In [21]:
```
lesson_table = pd.DataFrame(long_term.lessons(active_only=False))
display(
    lesson_table[["key", "count", "tickers", "text"]] if not lesson_table.empty els
)

print("Context the next NVDA run will receive from long-term memory:")
print(json.dumps(long_term.context_for("NVDA"), indent=2)[:3000])
```

	key	count	tickers	text
0	grounding_money	2	[AAPL, NVDA]	Do not write price targets, entry or stop-loss prices, support/resistance levels or do...
1	grounding_multiple	1	[MSFT]	Quote only valuation multiples (P/E, EV/Revenue, ...) that appear in the retrieved dat...
2	grounding_percent	1	[NVDA]	Write a calculated percentage as '≈N%' on the same line as one of its inputs, and chec...
3	grounding_other	1	[NVDA]	Copy every figure exactly as it appears in the retrieved data.

```
Context the next NVDA run will receive from long-term memory:
{
  "ticker_notes": [
    {
      "date": "2026-10-18T05:32:22+00:00",
      "question": "Provide an institutional equity research briefing on NVDA (NVIDIA
Corporation) covering fundamentals, valuation, technical momentum, and news catalyst
s.",
      "stance": "Bullish",
      "first_draft_score": 8.71,
      "final_score": 8.8,
      "passed": false,
      "refinements": 1,
      "first_draft_unverified": 5,
      "open_issues": [
        "2. Technical Profile & Market Momentum; 4. Investment Stance & Outlook: fig
ures not found in the retrieved data: 2. Technical Profile & Market Momentum: '5.7
%'",
        "Investment Stance & Outlook: Target\u2011price range ($240\u2011$250) and s
top\u2011loss level are presented without any grounding in the provided data, violat
ing the rule aga",
        "Fundamental & Earnings Performance \u2013 Balance: The brief leans heavily
toward a bullish narrative; the bearish side is limited to a list of risks without a
cohes"
      ]
    }
  ],
  "critic_lessons": [
    "Do not write price targets, entry or stop-loss prices, support/resistance level
s or dollar amounts that are not in the retrieved data."
  ],
  "usage": "Long-term memory from earlier runs. Ticker notes are dated hints, never
facts: always fetch fresh data and never copy figures from them. Follow the critic l
essons when writing the brief."
}
```

10.3 NVDA again, with what was learned

A brand-new conversation, so nothing is cached: the only difference from the first NVDA run is long-term memory (the active lessons and the note from the first NVDA run, including its open issues). The comparison focuses on the **first draft**, the text the lessons are meant to improve.

In [22]:
```python
# Rerun NVDA in a new conversation; long-term memory is the only thing carried over
context = long_term.context_for("NVDA")
lessons_used = len(context.get("critic_lessons", []))
notes_used = len(context.get("ticker_notes", []))
issues_before = long_term.notes("NVDA")[-1]["open_issues"]

rerun_session = ask_researched(
    lambda attempt: ResearchSession(
        thread_id=f"walkthrough-nvda-rerun-{attempt}-{int(time.time())}", long_term
    ),
    main_question,
```

```
)
main_stats = round_stats(main_state)
rerun_stats = round_stats(rerun_session.last_state)
issues_after = long_term.notes("NVDA")[-1]["open_issues"]

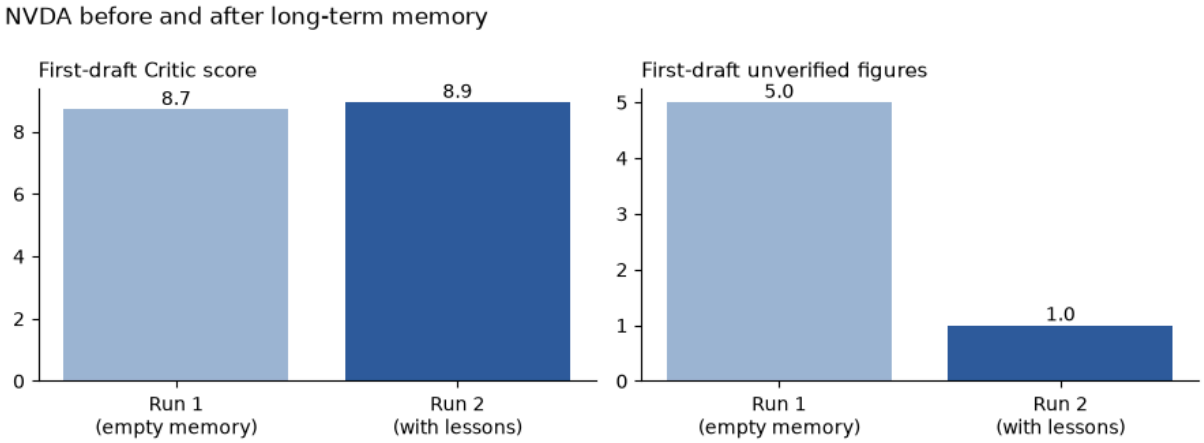
comparison = pd.DataFrame(
    [
        {
            "Run": "NVDA run 1 (empty memory)",
            "Active lessons used": 0,
            "Earlier notes used": 0,
            "First-draft score": main_stats["first_draft"],
            "First-draft unverified": main_stats["first_unverified"],
            "Released score": main_stats["released"],
            "Refinements": main_stats["refinements"],
            "Open issues at release": len(issues_before),
        },
        {
            "Run": "NVDA run 2 (with memory)",
            "Active lessons used": lessons_used,
            "Earlier notes used": notes_used,
            "First-draft score": rerun_stats["first_draft"],
            "First-draft unverified": rerun_stats["first_unverified"],
            "Released score": rerun_stats["released"],
            "Refinements": rerun_stats["refinements"],
            "Open issues at release": len(issues_after),
        },
    ]
)
display(comparison)
print("Open issues carried over from run 1:", issues_before or "none")
print("Open issues left after run 2       :", issues_after or "none")
```

	Run	Active lessons used	Earlier notes used	First-draft score	First-draft unverified	Released score	Refinements	Open issues at release
0	NVDA run 1 (empty memory)	0	0	8.71	5	8.80	1	3
1	NVDA run 2 (with memory)	1	1	8.94	1	9.15	1	0

```
Open issues carried over from run 1: ["2. Technical Profile & Market Momentum; 4. In
vestment Stance & Outlook: figures not found in the retrieved data: 2. Technical Pro
file & Market Momentum: '5.7 %'", 'Investment Stance & Outlook: Target-price range
($240-$250) and stop-loss level are presented without any grounding in the provided
data, violating the rule aga', 'Fundamental & Earnings Performance – Balance: The br
ief leans heavily toward a bullish narrative; the bearish side is limited to a list
of risks without a cohes']
Open issues left after run 2       : none
```

In [23]:
```
import matplotlib.pyplot as plt
```

```
fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
labels = ["Run 1\n(empty memory)", "Run 2\n(with lessons)"]
for ax, column, title in [
    (axes[0], "First-draft score", "First-draft Critic score"),
    (axes[1], "First-draft unverified", "First-draft unverified figures"),
]:
    bars = ax.bar(labels, comparison[column], color=["#9fb4d6", "#2f5d9c"])
    ax.bar_label(bars, fmt="%.1f")
    ax.set_title(title, loc="left", fontsize=11)
    ax.spines[["top", "right"]].set_visible(False)
fig.suptitle("NVDA before and after long-term memory", x=0.01, ha="left")
fig.tight_layout()
```



LLM output varies between runs, so a single comparison shows the mechanism rather than proving a fixed improvement; section 15 measures it over several tickers.

11. Conversation memory and edge cases

`ResearchSession` keeps each conversation under a `thread_id` in SQLite: the active company, recent turns, a rolling summary and the last brief for each ticker. A follow-up question about the same company is answered from the cached brief instead of running the research again.

The first cell asks a follow-up in the section 6 conversation. The second sends inputs that must be handled without research: a pronoun follow-up, a mistyped ticker, an off-topic request and an empty message.

In [24]:
```
# A follow-up in the same conversation: answered from the cached NVDA brief.
follow_up = "What were NVIDIA's exact gross and operating margins, and its forward
answer = main_session.ask(follow_up, show=False)
nodes = [step["node"] for step in main_session.last_state.trajectory]

print(f"Turn kind        : {main_session.last_kind}")
print(f"Active ticker    : {main_session.active_ticker}")
print(f"Nodes that ran   : {nodes}")
print(f"Research re-run  : {'orchestrator' in nodes}")
```

```
print("\n--- Answer from conversation memory ---")
show_md(answer)
```

```
Turn kind        : general
Active ticker    : NVDA
Nodes that ran   : ['planner']
Research re-run  : False

--- Answer from conversation memory ---
```

NVIDIA's reported metrics are:

- **Gross margin:** 74.98%
- **Operating margin:** 66.24%
- **Forward P/E ratio:** 13.45 ×

In [25]:
```
cases = [
    ("Pronoun follow-up (uses the cached NVDA brief)", "What about its margins?"),
    ("Typo in a ticker", "Analyse NVDAA"),
    ("Off-topic request", "What's the weather in Tokyo?"),
    ("Empty input", ""),
]
rows = []
for label, question in cases:
    answer = main_session.ask(question, show=False)
    rows.append(
        {
            "Case": label,
            "Input": question or "(empty)",
            "Turn kind": str(main_session.last_kind),
            "Active ticker": main_session.active_ticker,
            "Answer (preview)": short(answer, 120),
        }
    )
pd.DataFrame(rows)
```

Out[25]:

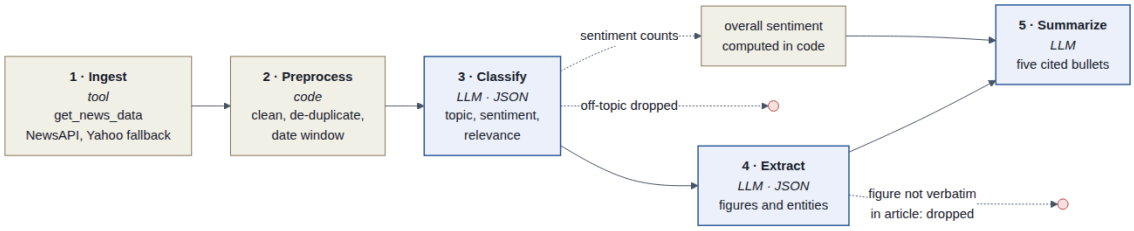
	Case	Input	Turn kind	Active ticker	Answer (preview)
0	Pronoun follow-up (uses the cached NVDA brief)	What about its margins?	follow_up	NVDA	NVIDIA’s reported margins are: - **Gross margin:** 74.98% - **Operating margin:** 66.24%
1	Typo in a ticker	Analyse NVDAA	unknown_ticker	NVDA	I couldn't find a stock with the ticker NVDAA. Did you mean NVDA?
2	Off-topic request	What's the weather in Tokyo?	general	NVDA	I’m an equity research assistant dedicated strictly to finance, stock analysis, and ma...
3	Empty input	(empty)	empty	NVDA	Please ask a question about a stock, for example: 'Analyse NVDA' or 'How did Apple's s...

Part III. Workflow patterns

12. W1 - Prompt chaining: the News analyst, stage by stage

The News analyst runs a five-stage prompt chain (`workflows/news_chain.py`). Each stage has one job and passes a structured result to the next. Code checks between stages stop invented headlines or numbers from reaching the next stage.

Stage	Done by	What happens
1. Ingest	tool	`get_news_data` : NewsAPI search for the company (Yahoo Finance news as fallback), last 30 days
2. Preprocess	code	clean the text, remove duplicates, keep only articles inside the window
3. Classify	LLM (JSON)	label each article's topic, sentiment and relevance; off-topic articles are dropped
4. Extract	LLM (JSON)	pull key figures and entities; a figure is kept only if it appears verbatim in the article
5. Summarize	LLM	five cited bullets; the overall sentiment is computed in code from stage 3



The cell prints the output of each stage.

In [26]:
```
news_analyst = news_analyst_agent(llm=llm)
news_response = news_analyst.run("Analyse NVDA")

for step in news_response.trace:
    stage = step.metadata.get("stage") or step.step_type
    if step.step_type == "tool_call":
        print(f"[Ingest] tool call: {step.content['tool']}({step.content['arguments
    elif step.step_type in ("tool_result", "observation", "thought", "error"):
        print(f"[{stage}] {short(step.content, 160)}")

print("\n=== Final news digest (stage 5 output) ===")
show_md(news_response.content)
```

```
[Ingest] tool call: get_news_data({'ticker': 'NVDA', 'days': 30})
[tool_result] { "ticker": "NVDA", "from": "2026-09-18", "to": "2026-10-18", "article
s": [ { "headline": "Nvidia says its new AI safety platform can contain rogue agents
...
[preprocess] {"stage": "preprocess", "result": {"received": 9, "kept": 9, "dropped":
{}}}
[classify] [ { "id": "A1", "topic": "deal_partnership", "sentiment": "Neutral", "rel
evance": 4 }, { "id": "A2", "topic": "product", "sentiment": "Bullish", ...
[classify] {"stage": "classify", "result": {"A1": "deal_partnership/Neutral/r4", "A
2": "product/Bullish/r4", "A3": "deal_partnership/Bullish/r3", "A4": ...
[filter_relevance] {"stage": "filter_relevance", "result": {"kept": 9, "dropped":
[]}}
[extract] [ { "id": "A1", "entities": [ "Nvidia", "UK" ], "figures": [ { "value":
"$250M", "context": "committed over five years to UK agentic safety and security ...
[extract] {"stage": "extract", "result": {"figures_kept": 9, "figures_rejected":
[]}}
[summarize] - Nvidia’s board approved an extra $150 billion for share buybacks, rais
ing total remaining authorization to $235 billion to be used through fiscal 2028,
...
[summarize_check] {"stage": "summarize_check", "result": {"bullets": 5, "filled_from
_headlines": 0}}

=== Final news digest (stage 5 output) ===
```

News digest: NVDA (last 30 days, 9 articles)

Overall news sentiment: Bullish (7 bullish, 1 neutral, 1 bearish) **Topics:** product (5), deal_partnership (2), regulation_legal (1), earnings (1)

Key developments

- Nvidia's board approved an extra $150 billion for share buybacks, raising total remaining authorization to $235 billion to be used through fiscal 2028, which should strengthen earnings per share and support the stock price. [A8]
- The company launched the Open Agent Safety Platform, a two-layer system that can contain rogue AI agents within milliseconds and is already adopted by over 100 organizations, positioning Nvidia as a leader in AI safety and opening new revenue opportunities. [A6][A7][A9]
- Nvidia pledged $250 million of compute resources over five years to UK agentic safety and security research, reinforcing its role in the AI safety ecosystem and potentially fostering future partnerships and technology adoption. [A1]
- Microsoft introduced a $2,599.99 Surface laptop powered by an Nvidia chip, claiming it doubles performance of a comparable MacBook Pro on local AI prompts, which could boost demand for Nvidia's AI hardware in consumer devices. [A2]
- Prosecutors allege a California man smuggled $300 million worth of Nvidia chips to China, a legal issue that may heighten regulatory scrutiny and pose reputational risk for the company. [A4]

Sources

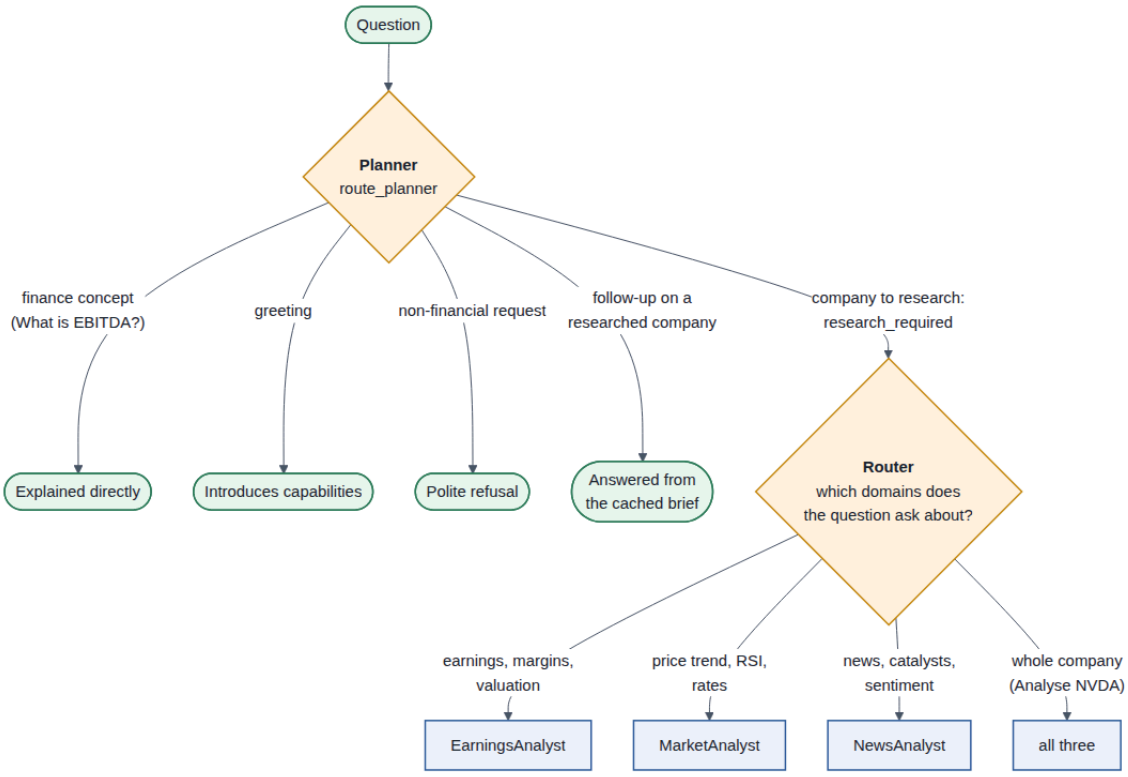
- [A1] Nvidia commits $250M of compute to UK AI safety research - The Next Web, 2026-10-14 (https://thenextweb.com/news/nvidia-250m-uk-ai-safety)
- [A2] Microsoft says new $2,599 laptop with Nvidia chip outperforms MacBooks - The Next Web, 2026-10-07 (https://thenextweb.com/news/surface-laptop-ultra-nvidia-europe)
- [A3] Nvidia-backed Reflection unveils Beam, its first open-weight model - The Next Web, 2026-10-05 (https://thenextweb.com/news/reflection-ai-beam-open-weight-model)
- [A4] California man smuggled $300 million in Nvidia chips to China, prosecutors say - Business Insider, 2026-10-02 (https://www.businessinsider.com/us-man-charged-smuggling-300-million-nvidia-chips-china-2026-10#article)
- [A5] Worried About AI Agents Destroying the World? Nvidia Has Software for That. - Entrepreneur, 2026-09-29 (https://www.entrepreneur.com/business-news/worried-about-ai-agents-destroying-the-world-nvidia-has-software-for-that)
- [A6] Nvidia says its new AI safety platform can contain rogue agents within 'milliseconds' - The Verge, 2026-09-28 (https://www.theverge.com/tech/1001287/nvidia-ai-safety-platform-rogue-agents)
- [A7] Nvidia launches agent safety platform backed by over 100 companies - The Next Web, 2026-09-28 (https://thenextweb.com/news/nvidia-open-agent-safety-platform)

- [A8] Nvidia adds a record $150bn to its share buyback - The Next Web, 2026-09-28 (https://thenextweb.com/news/nvidia-150bn-share-buyback-235bn-record)
- [A9] Nvidia launched a tool designed to stop AI agents from going rogue. Here's how it works. - Business Insider, 2026-09-28 (https://www.businessinsider.com/nvidia-launches-open-agent-safety-platform-ai-going-rogue-2026-9#article)

13. W2 - Routing: each request goes to the right place

Routing happens at two levels:

- **Planner route** (`route_planner` in `workflows/graph.py`): a research request goes to the orchestrator; a general question, greeting or off-topic request is answered directly (END).
- **Router** (`ResearchRouter`): decides *which* specialists a research request needs. A question only about earnings goes to the Earnings analyst; "Analyse NVDA" goes to all three.



The table runs the planner on eight test inputs and, for research requests, records the specialists the router delegates to (the specialists themselves are not run for this table).

In [27]:
```
from research_agent.agents import (
    earnings_analyst_agent,
    market_analyst_agent,
    news_analyst_agent,
```

```
    router_agent,
)
from research_agent.workflows.graph import route_planner

router = router_agent(
    llm=llm,
    subagents=[
        earnings_analyst_agent(llm=llm),
        market_analyst_agent(llm=llm),
        news_analyst_agent(llm=llm),
    ],
    verbose=False,
)


SPECIALIST_NAMES = {
    "earningsanalyst": "Earnings",
    "marketanalyst": "Market",
    "newsanalyst": "News",
}


def router_choice(question: str, max_turns: int = 4) -> list[str]:
    """Specialists the router delegates to, without running them.

    The router is a ReAct agent that may delegate one specialist per turn, so each
    delegation gets a placeholder result and the router is asked again until it sto
    """
    messages = [
        {"role": "system", "content": router._format_system_prompt()},
        {"role": "user", "content": question},
    ]
    chosen: list[str] = []
    for _ in range(max_turns):
        _, calls, raw_message = llm.chat(messages, router.tools)
        if not calls:
            break
        messages.append(raw_message)
        for call in calls:
            # Only delegations count; any other tool name the model invents is igno
            name = SPECIALIST_NAMES.get(call["name"].replace("delegate_to_", ""))
            if name and call["name"].startswith("delegate_to_"):
                chosen.append(name)
            messages.append({"role": "tool", "content": "(routing demo: not run)"})
    return list(dict.fromkeys(chosen)) or ["(answered directly)"]


test_inputs = [
    "Analyse NVDA",
    "How are Apple's latest earnings and margins?",
    "What is the price trend and RSI for Tesla?",
    "Any recent news catalysts for Exxon Mobil?",
    "What is EBITDA?",
    "Hi, what can you do?",
    "What's the weather in Tokyo?",
    "Compare JPMorgan's fundamentals and news sentiment",
```

```
]

rows = []
for question in test_inputs:
    response = planner_agent(llm=llm, tools=GraphMemory().as_tools()).run(question)
    state = GraphState(question=question)
    state.agents["ResearchPlanner"] = response.content
    route = route_planner(state)
    data = response.data or {}
    rows.append(
        {
            "Input": question,
            "Planner route": "research (orchestrator)" if route != END else "answer
            "Ticker": data.get("ticker", "-"),
            "Router: specialists": ", ".join(router_choice(question)) if route != E
        }
    )

routing_table = pd.DataFrame(rows)
routing_table
```

Out[27]:

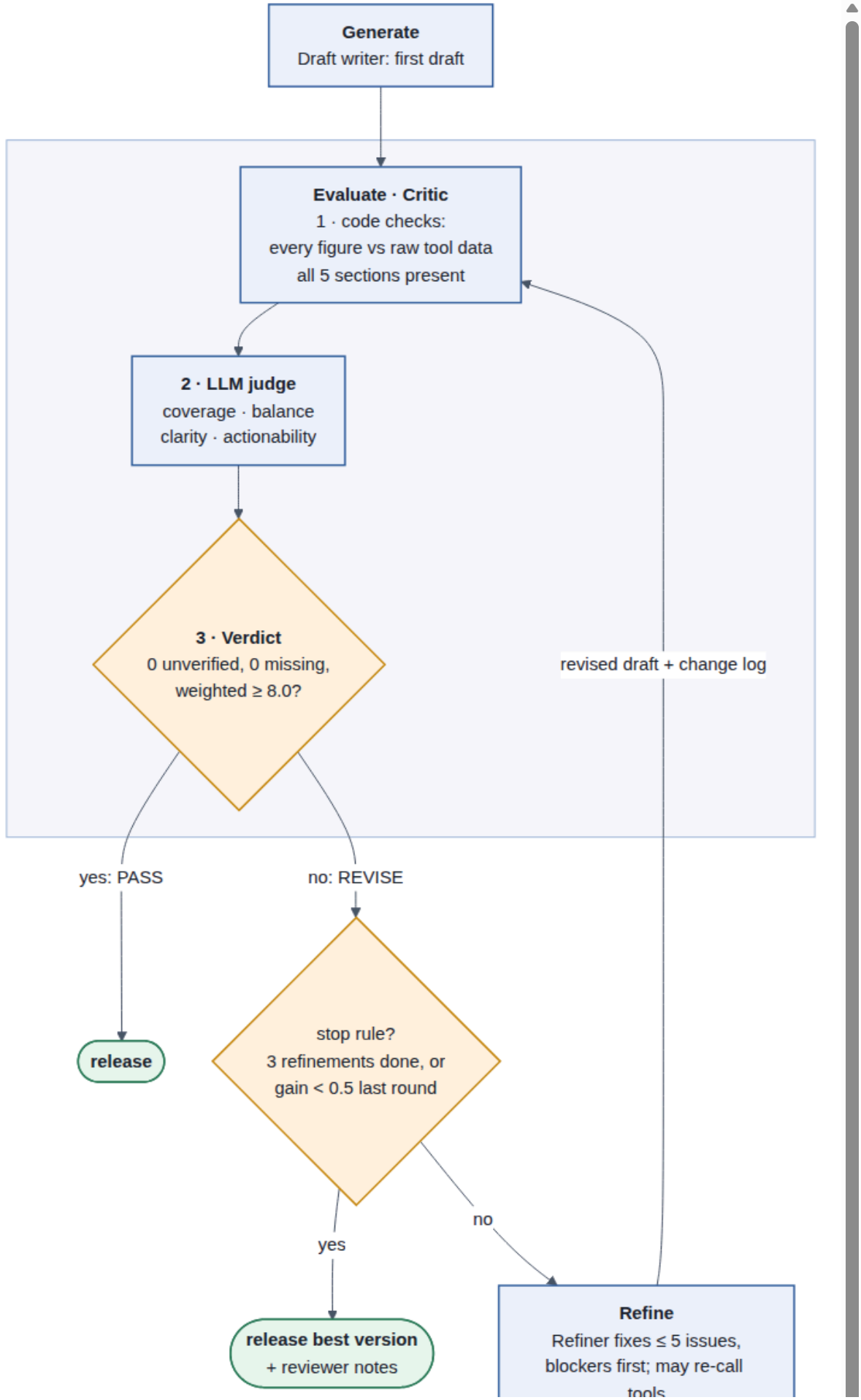
	Input	Planner route	Ticker	Router: specialists
0	Analyse NVDA	research (orchestrator)	NVDA	Earnings, Market, News
1	How are Apple's latest earnings and margins?	research (orchestrator)	AAPL	Earnings
2	What is the price trend and RSI for Tesla?	research (orchestrator)	TSLA	Market
3	Any recent news catalysts for Exxon Mobil?	research (orchestrator)	XOM	News
4	What is EBITDA?	answer directly	-	-
5	Hi, what can you do?	answer directly	-	-
6	What's the weather in Tokyo?	answer directly	-	-
7	Compare JPMorgan's fundamentals and news sentiment	research (orchestrator)	JPM	Earnings, News

14. W3 - Evaluator-optimizer: generate, evaluate, refine

The Draft writer generates, the Critic evaluates (section 9) and the **Refiner** improves the draft using the Critic's feedback:

- It rewrites only the flagged sections, at most five issues per round, blockers first, and may call the data tools again to replace an unsupported figure.
- It returns a change log, and the revised draft goes back to the Critic.

- The loop stops on a PASS, after 3 refinements, or when a round improves the score by less than 0.5. `release` then returns the best-scoring version, with reviewer notes if none passed.


Generate
Draft writer: first draft
Evaluate · Critic
1 · code checks:
every figure vs raw tool data
all 5 sections present
2 · LLM judge
coverage · balance
clarity · actionability
3 · Verdict
0 unverified, 0 missing,
weighted ≥ 8.0?
revised draft + change log
yes: PASS
no: REVISE
release
stop rule?
3 refinements done, or
gain < 0.5 last round
yes
no
release best version
+ reviewer notes
Refine
Refiner fixes ≤ 5 issues,
blockers first; may re-call
tools

The cells below show what the Refiner changed between the first two versions, the score per round, and a test that the evaluator catches a planted wrong number.

14.1 What the Refiner changed

Lines removed (`-`) and added (`+`) between the first draft and the first refinement.

In [28]:
```
# Draft v1 -> v2: what the Refiner changed (unified diff of the first two versions)
import difflib

if len(history) > 1:
    diff = difflib.unified_diff(
        history[0]["brief"].splitlines(),
        history[1]["brief"].splitlines(),
        "draft v1",
        "draft v2",
        lineterm="",
        n=0,
    )
    changed = [line for line in diff if line.startswith(("+", "-"))]
    print("\n".join(short(line, 150) for line in changed[:40]))
    print(f"\n({len(changed)} changed lines in total)")
else:
    print("The first draft passed review, so no refinement was needed.")
```

```
--- draft v1
+++ draft v2
-**High-level stance:** **Bullish** – the combination of explosive top-line growth,
premium profitability, strong cash generation, and a ...
+**High-level stance:** **Bullish** – the combination of explosive top-line growth,
premium profitability, strong cash generation, and a ...
-- Forward P/E **≈13.5×**, modest given >100 % YoY growth and >60 % operating margi
n.
-- EV-based multiples (EV/EBITDA, EV/Revenue) could not be computed because the data
service did not supply debt or cash figures.
+- Forward P/E **≈13.5×**, modest given >100 % YoY growth and **66 % operating margi
n**.
+- EV-based multiples (EV/EBITDA, EV/Revenue) could not be computed because the data
service did not supply debt or cash figures.
-- **Customer concentration:** Two unnamed customers represent a material share of r
evenues.
+- **Customer concentration:** Two unnamed customers represent a material share of r
evenues; the filing does not disclose the exact percentage.
-- Price is **≈4 % below the 50-day SMA** ( (222.96-214.13)/222.96 ≈ 0.0396 ).
-- Price is **≈5.7 % above the 200-day SMA** ( (214.13-202.66)/202.66 ≈ 0.0566 ).
+- Price is **≈4 % below the 50-day SMA**.
+- Price is **≈5.7 % above the 200-day SMA**.
-- **Upside target:** $235 – $250 (near prior 52-week high).
+- **Upside target:** $240 – $250 (near prior 52-week high).
-*All figures and narrative are taken directly from the specialist agents' outputs;
no external estimates or extrapolations have been introduced.*
+*All figures and narrative are taken directly from the specialist agents' outputs;
no external estimates or extrapolations have been introduced.*
+
+---

(20 changed lines in total)
```

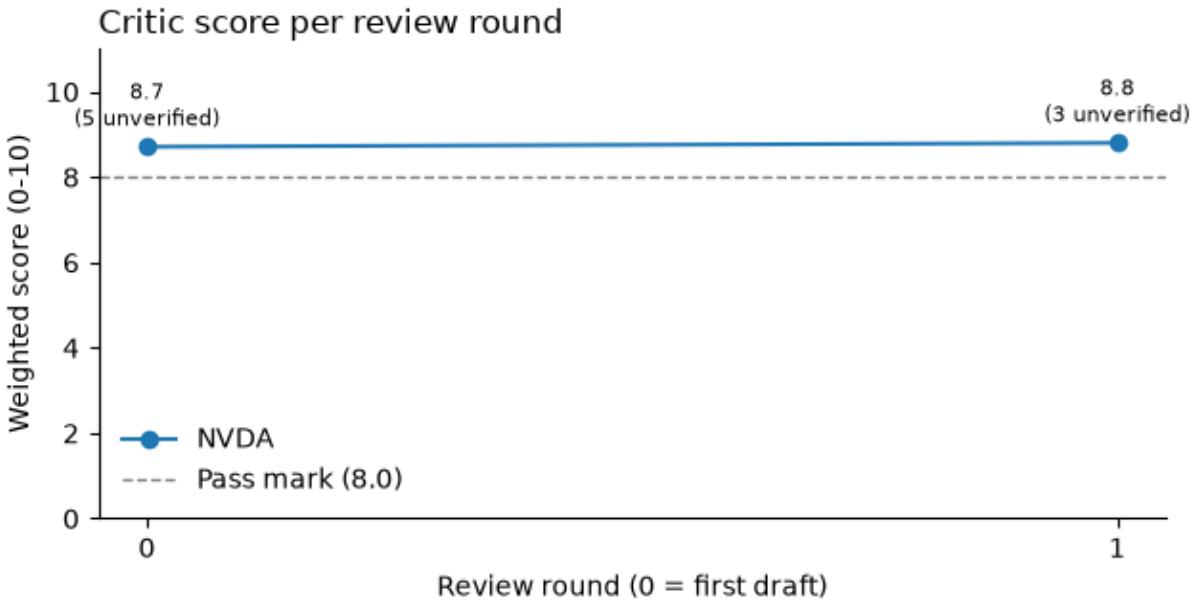
14.2 Score per review round

The weighted Critic score of each version, with the number of unverified figures. The dashed line is the pass mark.

In [29]:
```python
# Critic score per review round for this run (the evaluator-optimizer loop at work)
import matplotlib.pyplot as plt

main_stats = round_stats(main_state)
fig, ax = plt.subplots(figsize=(6.5, 3.4))
ax.plot(range(len(main_stats["scores"])), main_stats["scores"], marker="o", label="
for i, (score, unverified) in enumerate(
    zip(main_stats["scores"], main_stats["unverified"], strict=True)
):
    ax.annotate(
        f"{score:.1f}\n({unverified} unverified)",
        (i, score),
        textcoords="offset points",
        xytext=(0, 8),
        ha="center",
        fontsize=8,
    )
```

```
ax.axhline(8.0, color="grey", linestyle="--", linewidth=1, label="Pass mark (8.0)")
ax.set_xticks(range(len(main_stats["scores"])))
ax.set_ylim(0, 11)
ax.set_xlabel("Review round (0 = first draft)")
ax.set_ylabel("Weighted score (0-10)")
ax.set_title("Critic score per review round", loc="left")
ax.spines[["top", "right"]].set_visible(False)
ax.legend(frameon=False)
fig.tight_layout()
```



14.3 Critic test: a planted wrong number is caught

Taking the released brief, we change one verified dollar figure and review it again. The code check must list the planted figure as unverified and the verdict must be REVISE (design doc §5.2).

In [30]:
```
from research_agent.workflows.review_loop import (
    Critic,
    _parse_figures,
    check_grounding,
    evidence_numbers,
)

released = main_state.output.split("\n---\n### Reviewer notes")[0]
evidence = evidence_numbers(main_state.tools)

# Plant a wrong number: take a figure the data verifies, preferring large dollar am
# then any dollar amount, then any other figure, and inflate it by 37%.
verified = [
    f
    for line in released.splitlines()
    for f in _parse_figures(line)
    if evidence.matches(f) is not None and re.search(r"\d", f.text)
]
target = next(
```

```
    (f for f in verified if f.kind == "money" and f.value >= 1e6),
    next((f for f in verified if f.kind == "money"), verified[0] if verified else N
)
if target is None:
    # The released brief quotes nothing the data verifies (a weak draft). Test the
    # one-line brief built from a real tool figure instead, so the check does not d
    money = [v for v, kind in evidence.figures if kind in ("money", "plain") and ab
    if not money:
        raise RuntimeError(
            f"The tool data holds no dollar figure ({len(evidence.figures)} numbers
            "so section 6 did not fetch usable data. Re-run section 6."
        )
    released = f"Reported revenue was ${abs(money[0]) / 1e9:.2f}B in the latest qua
    target = next(f for f in _parse_figures(released) if evidence.matches(f) is not
    print(f"No verified figure in the released brief; testing on: {released}")
number = re.search(r"\d[\d,]*(?:\.\d+)?", target.text).group(0)
wrong = f"{float(number.replace(',', '')) * 1.37:.2f}"
assert number in released
planted_brief = released.replace(number, wrong, 1)

before = check_grounding(released, evidence).unverified
report = Critic(llm).review(planted_brief, main_state.tools, main_question, round_n
newly_flagged = [f for f in report.unverified if f not in before]
print(f"Planted change       : {number} -> {wrong}  (in '{target.text}')")
print(f"Newly unverified     : {newly_flagged}")
print(f"Grounding score      : {report.scores['grounding']}/10")
print(f"Verdict              : {'PASS' if report.passed else 'REVISE'}")
assert any(wrong in f for f in newly_flagged), "the planted figure must be flagged"
assert not report.passed, "the Critic must block a brief with a planted wrong numbe
```

```
Planted change       : 150 -> 205.50  (in '$150 bn')
Newly unverified     : ['$205.50 bn']
Grounding score      : 9.3/10
Verdict              : REVISE
```

Part IV. Evaluation and iteration

15. Evaluation

End-to-end evaluation across several tickers (design doc §5.1): Critic score per refinement round, pass rate, unverified figures before and after refinement, and run time.

15.1 Five-Ticker Evaluation

The function below researches five companies from different sectors, each in its own new conversation, and records the Critic score of every review round. Two charts follow: the score per refinement round for each ticker, and the final (released) score per ticker.

Ticker	Company	Sector
AAPL	Apple	Consumer technology

Ticker	Company	Sector
NVDA	NVIDIA	Semiconductors
JPM	JPMorgan Chase	Financials
XOM	Exxon Mobil	Energy
TSLA	Tesla	Automotive / EV

In [31]:
```
import contextlib
import tempfile
import threading

import matplotlib.pyplot as plt

from research_agent.memory import LongTermMemory

EVAL_TICKERS = {
    "AAPL": ("Apple", "Consumer technology"),
    "NVDA": ("NVIDIA", "Semiconductors"),
    "JPM": ("JPMorgan Chase", "Financials"),
    "XOM": ("Exxon Mobil", "Energy"),
    "TSLA": ("Tesla", "Automotive / EV"),
}
PASS_MARK = 8.0
# ---------------------------------------------------------------------------
# Latency and token usage per run
# ---------------------------------------------------------------------------
# The LLM wrappers return only the message, so token counts are read from the raw S
# responses by wrapping each client's call once. Ollama reports prompt_eval_count a
# eval_count; OpenAI-compatible clients (the Groq fallback) report usage.prompt_tok
# usage.completion_tokens.
# ASSUMED prices in USD per 1M tokens: replace them with your provider's rates. Oll
# is flat-rate, so the cost column is an estimate at these rates, not a bill.
PRICE_PER_1M_TOKENS = {"input": 0.15, "output": 0.60}

_usage_lock = threading.Lock()
_usage = {"calls": 0, "input": 0, "output": 0}


def _record_usage(prompt_tokens, completion_tokens):
    with _usage_lock:
        _usage["calls"] += 1
        _usage["input"] += int(prompt_tokens or 0)
        _usage["output"] += int(completion_tokens or 0)


def _wrap_call(owner, attribute, reader):
    """Wrap ``owner.attribute`` once so every response is counted."""
    original = getattr(owner, attribute)
    if getattr(original, "_usage_wrapped", False):
        return

    def counted(*args, **kwargs):
        response = original(*args, **kwargs)
```

```
        with contextlib.suppress(Exception):
            _record_usage(*reader(response))
        return response

    counted._usage_wrapped = True
    setattr(owner, attribute, counted)


def _install_usage_counter():
    from research_agent.interfaces import llm as llm_module

    for cls, attach in (
        (llm_module.OllamaLLM, lambda client: _wrap_call(
            client, "chat",
            lambda r: (getattr(r, "prompt_eval_count", 0), getattr(r, "eval_count",
        (llm_module.OpenAILLM, lambda client: _wrap_call(
            client.chat.completions, "create",
            lambda r: (r.usage.prompt_tokens, r.usage.completion_tokens))),
    ):
        original_chat = cls.chat
        if getattr(original_chat, "_usage_wrapped", False):
            continue

        def chat(self, *args, _orig=original_chat, _attach=attach, **kwargs):
            with contextlib.suppress(Exception):
                _attach(self.client)  # no-op after the first call on this client
            return _orig(self, *args, **kwargs)

        chat._usage_wrapped = True
        cls.chat = chat


_install_usage_counter()


class TrackUsage:
    """Context manager: wall-clock latency and LLM tokens used inside the ``with``

    After the block, ``.seconds``, ``.calls``, ``.input_tokens``, ``.output_tokens`
    ``.cost`` (USD at PRICE_PER_1M_TOKENS) are set. Do not run two tracked blocks a
    """

    def __enter__(self):
        with _usage_lock:
            self._start = dict(_usage)
        self._t0 = time.time()
        return self

    def __exit__(self, *exc):
        self.seconds = time.time() - self._t0
        with _usage_lock:
            end = dict(_usage)
        self.calls = end["calls"] - self._start["calls"]
        self.input_tokens = end["input"] - self._start["input"]
        self.output_tokens = end["output"] - self._start["output"]
        self.cost = (
```

```
            self.input_tokens * PRICE_PER_1M_TOKENS["input"]
            + self.output_tokens * PRICE_PER_1M_TOKENS["output"]
        ) / 1e6
        return False


EVAL_STATES = {}  # symbol -> final GraphState, filled by evaluate_tickers (used in


def research_question(symbol, company):
    return (
        f"Provide an institutional equity research briefing on {symbol} ({company})
        "covering fundamentals, valuation, technical momentum, and news catalysts."
    )


def ask_until_researched(symbol, company, memory_factory, attempts=3, tag="eval"):
    """Ask for a brief, retrying when the Planner answers in prose instead of dispa

    The Planner is an LLM. Now and then it replies with a plain-text plan instead o
    dispatch, the graph then ends at the Planner, and no Critic review exists. A re
    conversation almost always dispatches. Latency and tokens of every attempt are
    Returns (final state, usage of all attempts).
    """
    with TrackUsage() as usage:
        for attempt in range(1, attempts + 1):
            session = ResearchSession(
                thread_id=f"{tag}-{symbol.lower()}-{attempt}-{int(time.time())}",
                long_term=memory_factory(),
            )
            session.ask(research_question(symbol, company), show=False)
            state = session.last_state
            if getattr(state, "review", None) and state.review.get("history"):
                return state, usage
            print(
                f"{symbol}: attempt {attempt}/{attempts} was not researched "
                "(Planner did not dispatch)"
            )
    return state, usage


def evaluate_tickers(tickers=EVAL_TICKERS, reuse_state=None, use_memory=False):
    """Research each ticker end to end, chart score per round and final score per t

    tickers     : {symbol: (company name, sector)}
    reuse_state : optional {symbol: GraphState} of runs already done (e.g. {"NVDA":
    use_memory  : False keeps every run independent of long-term lessons (a fair co
    Returns a DataFrame with one row per ticker.
    """
    reuse_state = reuse_state or {}
    rows, histories = [], {}
    for symbol, (company, sector) in tickers.items():
        if symbol in reuse_state:
            state, elapsed, usage = reuse_state[symbol], float("nan"), None
        else:
            # long_term=None would fall back to the shared default store, so a thro
```

```
            # directory keeps every run free of earlier lessons.
            try:
                state, usage = ask_until_researched(
                    symbol,
                    company,
                    lambda: long_term if use_memory else LongTermMemory(tempfile.mk
                )
            except Exception as exc:  # one failing ticker must not stop the evalua
                print(f"{symbol}: run failed ({type(exc).__name__}: {exc})")
                continue
            elapsed = usage.seconds
        if not getattr(state, "review", None) or not state.review.get("history"):
            print(f"{symbol}: no review history after all attempts (request was not
            continue
        EVAL_STATES[symbol] = state
        stats = round_stats(state)
        histories[symbol] = stats
        rows.append(
            {
                "Ticker": symbol,
                "Sector": sector,
                "Rounds": len(stats["scores"]),
                "First-draft score": stats["first_draft"],
                "Final score": stats["released"],
                "Gain": round(stats["released"] - stats["first_draft"], 2),
                "Unverified (first -> last)": (
                    f"{stats['unverified'][0]} -> {stats['unverified'][-1]}"
                ),
                "Passed": stats["passed"],
                "Latency (s)": round(elapsed, 1) if elapsed == elapsed else None,
                "LLM calls": usage.calls if usage else None,
                "Input tokens": usage.input_tokens if usage else None,
                "Output tokens": usage.output_tokens if usage else None,
                "Est. cost ($)": round(usage.cost, 4) if usage else None,
            }
        )
    table = pd.DataFrame(rows)
    if table.empty:
        print("No ticker produced a review history.")
        return table

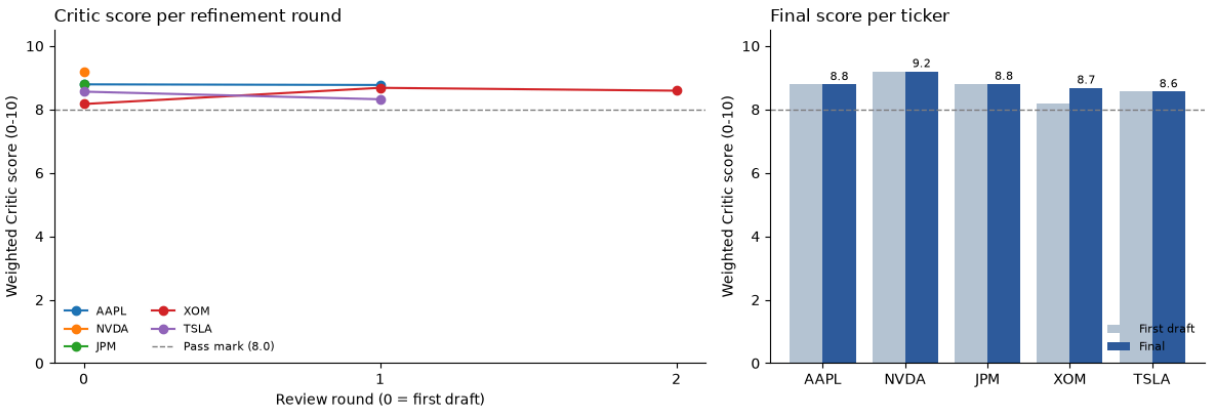
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.2), gridspec_kw={"width_rat

    # Chart 1: Critic score per refinement round, one line per ticker.
    for symbol, stats in histories.items():
        ax1.plot(range(len(stats["scores"])), stats["scores"], marker="o", label=sy
    ax1.axhline(
        PASS_MARK, color="grey", linestyle="--", linewidth=1, label=f"Pass mark ({P
    )
    max_rounds = max(len(s["scores"]) for s in histories.values())
    ax1.set_xticks(range(max_rounds))
    ax1.set_ylim(0, 10.5)
    ax1.set_xlabel("Review round (0 = first draft)")
    ax1.set_ylabel("Weighted Critic score (0-10)")
    ax1.set_title("Critic score per refinement round", loc="left")
    ax1.legend(frameon=False, fontsize=8, ncol=2)
```

```
    # Chart 2: final (released) score per ticker, with the first draft for referenc
    x = range(len(table))
    ax2.bar(
        [i - 0.2 for i in x], table["First-draft score"],
        width=0.4, color="#b8c4d6", label="First draft",
    )
    bars = ax2.bar(
        [i + 0.2 for i in x], table["Final score"], width=0.4, color="#2f5d9e", lab
    )
    for bar, score in zip(bars, table["Final score"], strict=True):
        ax2.annotate(f"{score:.1f}", (bar.get_x() + bar.get_width() / 2, score),
                     textcoords="offset points", xytext=(0, 3), ha="center", fontsi
    ax2.axhline(PASS_MARK, color="grey", linestyle="--", linewidth=1)
    ax2.set_xticks(list(x))
    ax2.set_xticklabels(table["Ticker"])
    ax2.set_ylim(0, 10.5)
    ax2.set_ylabel("Weighted Critic score (0-10)")
    ax2.set_title("Final score per ticker", loc="left")
    ax2.legend(frameon=False, fontsize=8, loc="lower right")

    for ax in (ax1, ax2):
        ax.spines[["top", "right"]].set_visible(False)
    fig.tight_layout()
    plt.show()
    display(table)
    return table


# All five tickers are researched in fresh conversations.
eval_table = evaluate_tickers()
```



	Ticker	Sector	Rounds	First-draft score	Final score	Gain	Unverified (first -> last)	Passed	Latency (s)	LLM calls	t
0	AAPL	Consumer technology	2	8.79	8.79	0.00	2 -> 1	False	86.9	22	
1	NVDA	Semiconductors	1	9.20	9.20	0.00	0 -> 0	True	65.9	19	
2	JPM	Financials	1	8.80	8.80	0.00	0 -> 0	True	70.4	19	
3	XOM	Energy	3	8.17	8.68	0.51	9 -> 6	False	107.8	28	1
4	TSLA	Automotive / EV	2	8.56	8.56	0.00	2 -> 4	False	76.8	22	

15.2 Accuracy check against yfinance

Each released brief is checked against fresh yfinance data using 10 evenly distributed figures covering company ratios, price history, and financial statements. Figures are **confirmed** if they match within rounding tolerance or 2%. Accuracy is the percentage of confirmed figures. Limitations: Agent-calculated figures and data from news or filings may lack matching yfinance fields, so “not found” does not mean incorrect. Matched fields are shown for manual verification.

In [32]:
```
import numpy as np
import yfinance as yf

from research_agent.workflows.review_loop import _parse_figures


def yfinance_reference(symbol):
    """Numbers yfinance reports for ``symbol``, as {field name: value}."""
    tk = yf.Ticker(symbol)
    ref = {}
    with contextlib.suppress(Exception):
        for key, value in (tk.info or {}).items():
            if isinstance(value, (int, float)) and not isinstance(value, bool):
                ref[f"info.{key}"] = float(value)
    with contextlib.suppress(Exception):
        hist = tk.history(period="1y")
        close, volume = hist["Close"].dropna(), hist["Volume"].dropna()
        ref.update(
            {
                "price.close": close.iloc[-1],
                "price.sma50": close.tail(50).mean(),
                "price.sma200": close.tail(200).mean(),
                "price.high_52w": close.max(),
                "price.low_52w": close.min(),
                "price.volume": volume.iloc[-1],
                "price.avg_volume_3m": volume.tail(63).mean(),
            }
```

```
        )
        delta = close.diff()
        gain, loss = delta.clip(lower=0).tail(14).mean(), -delta.clip(upper=0).tail
        ref["price.rsi14"] = 100 - 100 / (1 + gain / loss) if loss else 100.0
    for name in (
        "quarterly_income_stmt",
        "income_stmt",
        "quarterly_cashflow",
        "cashflow",
        "quarterly_balance_sheet",
        "balance_sheet",
    ):
        with contextlib.suppress(Exception):
            frame = getattr(tk, name)
            for column in list(frame.columns)[:2]:  # latest two periods
                for row, value in frame[column].dropna().items():
                    ref[f"{name}.{row}[{str(column)[:10]}]"] = float(value)
    return {k: float(v) for k, v in ref.items() if v == v}

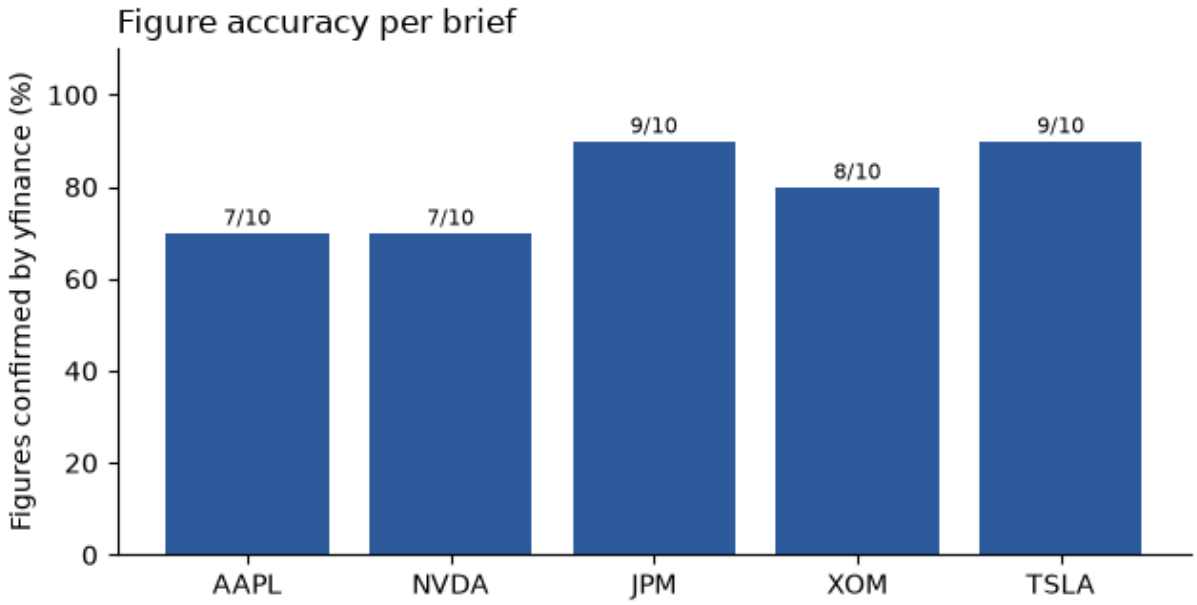

def match_reference(figure, ref, rel_tol=0.02):
    """Name of the yfinance field that ``figure`` agrees with, or None."""
    target = abs(figure.value)
    allowed = max(figure.tolerance, rel_tol * target)
    # A percentage may be stored as a fraction (0.62 -> 62%).
    scales = (1.0, 100.0) if figure.kind == "percent" else (1.0,)
    best = None
    for name, value in ref.items():
        for scale in scales:
            gap = abs(target - abs(value) * scale)
            if gap <= allowed and (best is None or gap < best[0]):
                best = (gap, name)
    return best[1] if best else None


def verify_figures(brief, symbol, n=10, ref=None):
    """Check ``n`` cited figures of ``brief`` against yfinance; returns one row per
    brief = brief.split("\n---\n### Reviewer notes")[0]
    ref = ref or yfinance_reference(symbol)
    unique, seen = [], set()
    for line in brief.splitlines():
        if line.lstrip().startswith("#"):
            continue
        for figure in _parse_figures(line):
            key = (figure.kind, round(figure.value, 6))
            if key not in seen:  # one entry per distinct value
                seen.add(key)
                unique.append((figure, line))
    if not unique:
        return pd.DataFrame()
    picks = [unique[i] for i in np.linspace(0, len(unique) - 1, min(n, len(unique))
    rows = []
    for figure, line in picks:
        field = match_reference(figure, ref)
        rows.append(
            {
```

```
                "Ticker": symbol,
                "Figure": figure.text,
                "Context": short(line.strip("|-* "), 70),
                "yfinance field": field or "-",
                "Confirmed": field is not None,
            }
        )
    return pd.DataFrame(rows)


def accuracy_report(states=None, n=10):
    """Run the yfinance check on every brief in ``states`` and chart accuracy per t
    states = states or EVAL_STATES
    if not states:
        print("No briefs yet: run the evaluation cell above first.")
        return pd.DataFrame()
    details = pd.concat(
        [verify_figures(state.output, symbol, n) for symbol, state in states.items(
        ignore_index=True,
    )
    summary = (
        details.groupby("Ticker", sort=False)["Confirmed"]
        .agg(Checked="size", Confirmed="sum")
        .assign(Accuracy=lambda d: (100 * d["Confirmed"] / d["Checked"]).round(0))
        .reset_index()
    )
    fig, ax = plt.subplots(figsize=(6.5, 3.4))
    bars = ax.bar(summary["Ticker"], summary["Accuracy"], color="#2f5d9e")
    for bar, row in zip(bars, summary.itertuples(), strict=True):
        ax.annotate(
            f"{row.Confirmed}/{row.Checked}",
            (bar.get_x() + bar.get_width() / 2, row.Accuracy),
            textcoords="offset points",
            xytext=(0, 3),
            ha="center",
            fontsize=8,
        )
    ax.set_ylim(0, 110)
    ax.set_ylabel("Figures confirmed by yfinance (%)")
    ax.set_title("Figure accuracy per brief", loc="left")
    ax.spines[["top", "right"]].set_visible(False)
    fig.tight_layout()
    plt.show()
    total = int(details["Confirmed"].sum())
    pct = 100 * total / len(details)
    print(f"Overall: {total} of {len(details)} figures confirmed ({pct:.0f}%)")
    display(summary)
    display(details)
    return details


accuracy_table = accuracy_report()
```




```
Overall: 40 of 50 figures confirmed (80%)
```

	Ticker	Checked	Confirmed	Accuracy
0	AAPL	10	7	70.0
1	NVDA	10	7	70.0
2	JPM	10	9	90.0
3	XOM	10	8	80.0
4	TSLA	10	9	90.0

	Ticker	Figure	Context	yfinance field	Confirmed
0	AAPL	$98.8 B	Financial strength:** $98.8 B annual free cash flow; net-cash ...	cashflow.Free Cash Flow[2025-09-30]	True
1	AAPL	1.6 %	Growth:** FY 2026 Q2 revenue $109.42 B, **+16.4 % YoY**, though ...	info.heldPercentInsiders	True
2	AAPL	$325.69	Technicals:** Price $343.46 trades **well above** the 50-day SMA ...	price.sma50	True
3	AAPL	1.59 %	QoQ Revenue Growth** | **-1.59 %** | EarningsAnalyst	-	False
4	AAPL	35.86×	Forward P/E** | **35.86×** | EarningsAnalyst	info.forwardPE	True
5	AAPL	$106,973	Marketable securities (total)** | $106,973 | SEC filing	-	False
6	AAPL	1.7 ×	Liquidity:** Ultra-liquid assets (cash + securities) > $146 B, > ...	-	False
7	AAPL	$243.42	52-week range** | $243.42 – $348.15 | Trading at ~99 % of the high	info.fiftyTwoWeekLow	True
8	AAPL	4.02 %	CPI YoY** | 4.02 % (Sep 2026)	info.askSize	True
9	AAPL	50 %	1. **Elite fundamentals:** 16 % YoY revenue growth, >50 % gross ...	info.twoHundredDayAverageChange	True
10	NVDA	$96.22 B	NVIDIA delivered **record-level Q2 FY 2026 results**, with revenue ...	quarterly_income_stmt.Total Revenue[2026-07-31]	True
11	NVDA	62 %	NVIDIA delivered **record-level Q2 FY 2026 results**, with revenue ...	-	False

	Ticker	Figure	Context	yfinance field	Confirmed
12	NVDA	37.6	Technically, the shares sit **8.9 % beneath the 50-day SMA** but ...	-	False
13	NVDA	66.24 %	Operating Margin**	**66.24 %** 【ResearchRouter†L5-L6】	...	info.operatingMargins	True
14	NVDA	$10.9 B	Long-term Liabilities**	**$10.9 B** 【ResearchRouter†L28-L31】	...	quarterly_balance_sheet.Net Debt[2026-07-31]	True
15	NVDA	39 %	Customer Concentration** | Two top customers: **23 %** and **16 ...	-	False
16	NVDA	$202.66	200-day SMA**	**$202.66** – price **5.6 % ...	price.sma200	True
17	NVDA	$164.27	52-wk high / low**	**$243.37 / $164.27** – current price **12.1 ...	info.fiftyTwoWeekLow	True
18	NVDA	$300 m	5 | **Alleged $300 m chip smuggling to China** (legal probe) | ...	info.sharesShortPriorMonth	True
19	NVDA	2.2 %	Real GDP Growth** | 2.2 % annualized (Apr 2026) | Moderate expansion	info.beta	True
20	JPM	$750 bn	Catalysts:** Large-scale housing-supply commitment ($750 bn) and ...	balance_sheet.Total Capitalization[2024-12-31]	True
21	JPM	$52.85 B	Revenue** | **$52.85 B	quarterly_income_stmt.Total Revenue[2026-06-30]	True
22	JPM	$5.75	EPS (actual)** | **$5.75	-	False
23	JPM	$902.2 B	Market Capitalisation** | **$902.2 B	info.marketCap	True
24	JPM	$346.69	50-day SMA** | **$346.69	price.sma50	True

	Ticker	Figure	Context	yfinance field	Confirmed
25	JPM	$366.50	52-wk high / low** | **$366.50 / $279.10	info.fiftyTwoWeekHigh	True
26	JPM	6 %	Price ≈6 % **above** the 200-day SMA (($339.72 − $319.46) / ...	info.auditRisk	True
27	JPM	3 %	1 | $20 bn sovereign-wealth partnership (stock fell 3 %) | Bearish ...	info.shareHolderRightsRisk	True
28	JPM	4.02 %	CPI YoY** | 4.02 %	info.compensationRisk	True
29	JPM	$320	Overall Bias** | **Cautiously bullish** – fundamentals and macro ...	price.sma200	True
30	XOM	$114.53 B	Revenue:** $114.53 B (Q2 2026) – **+44 % YoY**, driven by higher ...	quarterly_income_stmt.Total Revenue[2026-06-30]	True
31	XOM	$23.61 B	Cash Generation:** FY 2025 free-cash-flow $23.61 B; Q2 2026 ...	cashflow.Free Cash Flow[2025-12-31]	True
32	XOM	15×	Valuation:** Forward P/E ≈ 15× (in line with historic ...	info.sourceInterval	True
33	XOM	$163	Overall Positioning:** **Neutral-to-Buy** – robust cash flow, ...	price.sma50	True
34	XOM	15.09×	Forward P/E** | 15.09× | — | `get_earnings_data`	info.forwardPE	True
35	XOM	5 %	DCF Sensitivity (high-level):** 5 % discount-rate reduction or ...	-	False
36	XOM	$111.98	52-wk range** | $111.98 – $179.16 | At top of range, strong ...	info.fiftyTwoWeekLow	True
37	XOM	3.75 %	Macro backdrop** | Fed Funds 3.75 %; 10-yr Treasury 5.1 %; CPI 4.0 ...	-	False
38	XOM	22 %	5 | **Strong price momentum** | Shares	info.numberOfAnalystOpinions	True

	Ticker	Figure	Context	yfinance field	Confirmed
			up **52 % YTD**, **22 %** ...		
39	XOM	$23.6 B	Fundamentals** | Revenue +44 % YoY, healthy margins, $23.6 B FY ...	cashflow.Free Cash Flow[2025-12-31]	True
40	TSLA	$28.24 B	Tesla (NASDAQ TSLA) continues to deliver strong top-line growth, ...	quarterly_income_stmt.Total Revenue[2026-06-30]	True
41	TSLA	1.4 %	Tesla (NASDAQ TSLA) continues to deliver strong top-line growth, ...	info.operatingMargins	True
42	TSLA	9 %	Technicals show the price at **$354.72**, sitting **≈ 1.3 %** ...	info.shareHolderRightsRisk	True
43	TSLA	3.95 %	Net margin** | **3.95 %	info.auditRisk	True
44	TSLA	$25,097 M	Automotive** | **$25,097 M** | **$4,111 M	quarterly_balance_sheet.Total Non Current Liabilities Net Minority Interest[2026-03-31]	True
45	TSLA	$146 M	Revenue composition*: automotive sales $20,006 M, regulatory ...	-	False
46	TSLA	$28,236 M	Revenue composition*: automotive sales $20,006 M, regulatory ...	quarterly_income_stmt.Total Revenue[2026-06-30]	True
47	TSLA	$9,637 M	Automotive inventory** | **$9,637 M** | Reflects build-up for new ...	quarterly_balance_sheet.Construction In Progress[2026-03-31]	True
48	TSLA	1.916	Beta (vs. S&P 500)** | **1.916	info.beta	True
49	TSLA	4.02 %	CPI YoY** | **4.02 %** (Sept 2026)	info.auditRisk	True

15.3 Long-term memory (F4): one ticker researched twice

The same ticker is researched twice in separate conversations: Run 1 starts with empty memory; Run 2 uses lessons from researching seed tickers and Run 1's dated note. First

drafts are compared before Critic feedback. Temporary memory preserves existing long-term memory. Due to LLM variability, the comparison illustrates the mechanism, not guaranteed improvement.

In [33]:
```
def memory_effect(ticker="NVDA", seed_tickers=("AAPL",)):
    """Research ``ticker`` twice (empty memory, then with memory) and compare first
    memory = LongTermMemory(tempfile.mkdtemp())

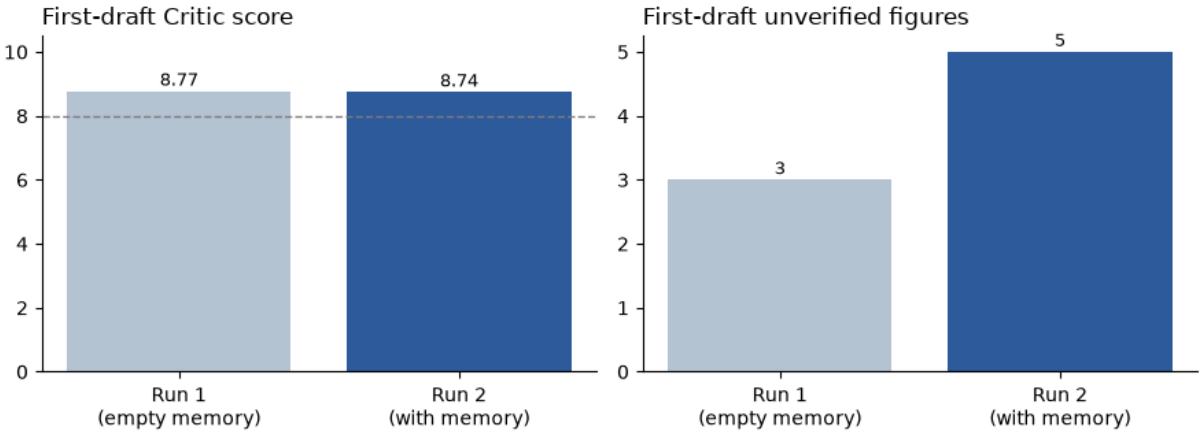
    def run(symbol, label):
        company = EVAL_TICKERS.get(symbol, (symbol,))[0]
        state, _ = ask_until_researched(symbol, company, lambda: memory, tag=f"mem-
        if not getattr(state, "review", None) or not state.review.get("history"):
            raise RuntimeError(f"{symbol} was not researched after retries; run the
        return state

    first = round_stats(run(ticker, "run1"))
    for symbol in seed_tickers:
        run(symbol, "seed")
    context = memory.context_for(ticker)
    lessons, notes = len(context.get("critic_lessons", [])), len(context.get("ticke
    second = round_stats(run(ticker, "run2"))

    table = pd.DataFrame(
        [
            {
                "Run": f"{ticker} run 1 (empty memory)",
                "Active lessons": 0,
                "Earlier notes": 0,
                "First-draft score": first["first_draft"],
                "First-draft unverified": first["first_unverified"],
                "Released score": first["released"],
            },
            {
                "Run": f"{ticker} run 2 (with memory)",
                "Active lessons": lessons,
                "Earlier notes": notes,
                "First-draft score": second["first_draft"],
                "First-draft unverified": second["first_unverified"],
                "Released score": second["released"],
            },
        ]
    )
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
    labels = ["Run 1\n(empty memory)", "Run 2\n(with memory)"]
    for ax, column, title in [
        (axes[0], "First-draft score", "First-draft Critic score"),
        (axes[1], "First-draft unverified", "First-draft unverified figures"),
    ]:
        bars = ax.bar(labels, table[column], color=["#b8c4d6", "#2f5d9e"])
        for bar, value in zip(bars, table[column], strict=True):
            ax.annotate(
                f"{value:g}",
                (bar.get_x() + bar.get_width() / 2, value),
                textcoords="offset points",
                xytext=(0, 3),
```

```
                ha="center",
                fontsize=9,
            )
        ax.set_title(title, loc="left")
        ax.spines[["top", "right"]].set_visible(False)
    axes[0].axhline(PASS_MARK, color="grey", linestyle="--", linewidth=1)
    axes[0].set_ylim(0, 10.5)
    fig.tight_layout()
    plt.show()
    display(table)
    delta = second["first_draft"] - first["first_draft"]
    print(f"First-draft score change with memory: {delta:+.2f}")
    return table


memory_table = memory_effect("NVDA")
```



	Run	Active lessons	Earlier notes	First-draft score	First-draft unverified	Released score
0	NVDA run 1 (empty memory)	0	0	8.77	3	8.82
1	NVDA run 2 (with memory)	2	1	8.74	5	8.85

```
First-draft score change with memory: -0.03
```

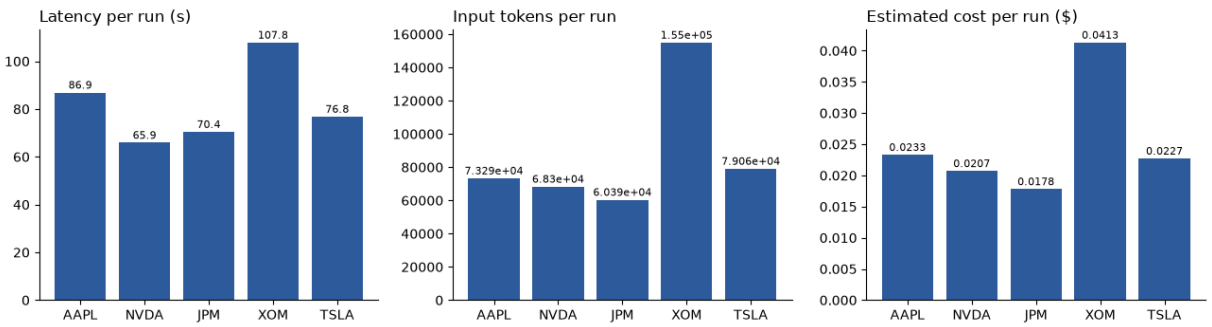
15.4 Latency and token cost per run

Each run tracks execution time and LLM input/output tokens. Costs use placeholder rates in `PRICE_PER_1M_TOKENS`; replace them with actual provider rates. Groq fallback usage is included, but unreported token usage counts as zero, so verify against the LLM call count.

In [34]:
```
def plot_latency_and_cost(table):
    """Bar charts of latency (s) and estimated cost ($) per ticker, plus tokens in
    data = table.dropna(subset=["Latency (s)"])
    if data.empty:
        print("No timed runs in the table.")
        return
    fig, axes = plt.subplots(1, 3, figsize=(13, 3.6))
```

```
    for ax, column, title in [
        (axes[0], "Latency (s)", "Latency per run (s)"),
        (axes[1], "Input tokens", "Input tokens per run"),
        (axes[2], "Est. cost ($)", "Estimated cost per run ($)"),
    ]:
        bars = ax.bar(data["Ticker"], data[column], color="#2f5d9e")
        for bar, value in zip(bars, data[column], strict=True):
            ax.annotate(f"{value:,.4g}", (bar.get_x() + bar.get_width() / 2, value)
                        textcoords="offset points", xytext=(0, 3), ha="center", fon
        ax.set_title(title, loc="left")
        ax.spines[["top", "right"]].set_visible(False)
    fig.tight_layout()
    plt.show()
    columns = [
        "Ticker", "Latency (s)", "LLM calls",
        "Input tokens", "Output tokens", "Est. cost ($)",
    ]
    totals = data[columns[1:]].sum()
    print(
        f"Total: {totals['Latency (s)']:.0f} s, {int(totals['LLM calls'])} LLM call
        f"{int(totals['Input tokens']):,} input + {int(totals['Output tokens']):,}
        f"about ${totals['Est. cost ($)']:.3f} at the assumed prices"
    )
    display(data[columns])


plot_latency_and_cost(eval_table)
```




```
Total: 408 s, 110 LLM calls, 436,011 input + 100,433 output tokens, about $0.126 at
the assumed prices
```

	Ticker	Latency (s)	LLM calls	Input tokens	Output tokens	Est. cost ($)
0	AAPL	86.9	22	73289	20466	0.0233
1	NVDA	65.9	19	68296	17418	0.0207
2	JPM	70.4	19	60388	14492	0.0178
3	XOM	107.8	28	154974	30026	0.0413
4	TSLA	76.8	22	79064	18031	0.0227

16. Iteration and code quality

The system improves its output at two time scales, both shown with real runs above:

Scale	Mechanism	Evidence in this notebook
Within one run	Critic and Refiner loop (W3): each round fixes the issues the Critic found	section 9 table and section 14 chart
Across runs	Long-term memory (F4): recurring first-draft weaknesses become lessons for the next draft	section 10 comparison

The code was also developed iteratively, with each fix covered by a test. The repository follows PEP 8 / PEP 257 (checked with `ruff`), uses type hints (`mypy`), and has an offline `pytest` suite with scripted LLMs and stubbed data tools (above 90% line coverage). The cell below runs the tests for the components demonstrated in this notebook.

In [35]:
```
test_files = [
    "tests/test_review_loop.py",
    "tests/test_long_term_memory.py",
    "tests/test_news_chain.py",
    "tests/test_router.py",
    "tests/test_session.py",
]
try:
    import pytest  # noqa: F401
except ImportError:
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "pytest"], check=

result = subprocess.run(
    [
        sys.executable,
        "-m",
        "pytest",
        "-q",
        "--color=no",
        "-o",
        "addopts=",
        "-p",
        "no:cacheprovider",
        "--basetemp=.pytest_temp",
        *test_files,
    ],
    capture_output=True,
    text=True,
    check=False,  # failures are reported in the printed summary
)
print(result.stdout.strip().splitlines()[-1])
```

```
130 passed in 6.86s
```

17. Summary

Requirement	How it is implemented	Demonstrated in
F1 Planning	Planner verifies the ticker with `lookup_ticker` and returns a JSON dispatch with an ordered research plan	Section 7
F2 Dynamic tool use	Specialists choose tools and arguments at run time (yfinance, SEC EDGAR, FRED, NewsAPI, Alpha Vantage fallback)	Sections 3, 6, 8
F3 Self-reflection	Critic: code grounding check against raw tool data + LLM judge on four dimensions	Section 9
F4 Learning across runs	Long-term memory: ticker notes and Critic lessons that feed later runs	Section 10
W1 Prompt chaining	News analyst: ingest, preprocess, classify, extract, summarize	Section 12
W2 Routing	Planner routes research vs direct answers; router picks the specialists	Section 13
W3 Evaluator-optimizer	Critic ⇄ Refiner loop with stopping rules and best-version release	Sections 9, 14

Limitations. The LLM judge's scores vary by about ±0.5 between runs, which can stop the loop early on a plateau; free-tier API limits apply (results are cached); the system covers US-listed equities. This is a research tool, not financial advice.

Repository: https://github.com/diaeshantony-usd/investment-research-agent

Team: Sai Krishna Akula (Planner, Router, Earnings analyst, orchestrator, CLI) · Ashok Kumar Bhairwal (data tools and caching, Market analyst, Gradio UI, tests, evaluation) · Diaesh Antony (News prompt chain, Draft writer, Critic, Refiner, session and long-term memory).

AI-assistance disclosure

In line with the course policy on AI tools, the team discloses that an AI coding assistant (Claude) was used to help draft, review and test parts of the code and of this notebook. All AI-assisted code was reviewed, run and tested by the team, and each component is explained in this notebook.